System for Artificial Intelligence

GPU Architecture & CUDA Programming

Siyuan Feng
Shanghai Innovation Institute

Learning Goals

上海创智学院
Shanghai Innovation Institute

Explain	GPU, SM, warp and memory
Program	Work assignment and safe cooperation
Optimize	Access patterns, reuse and measurement

OUTLINE

01 GPU Architecture

02 CUDA Programming

03 Memory Access

04 Performance

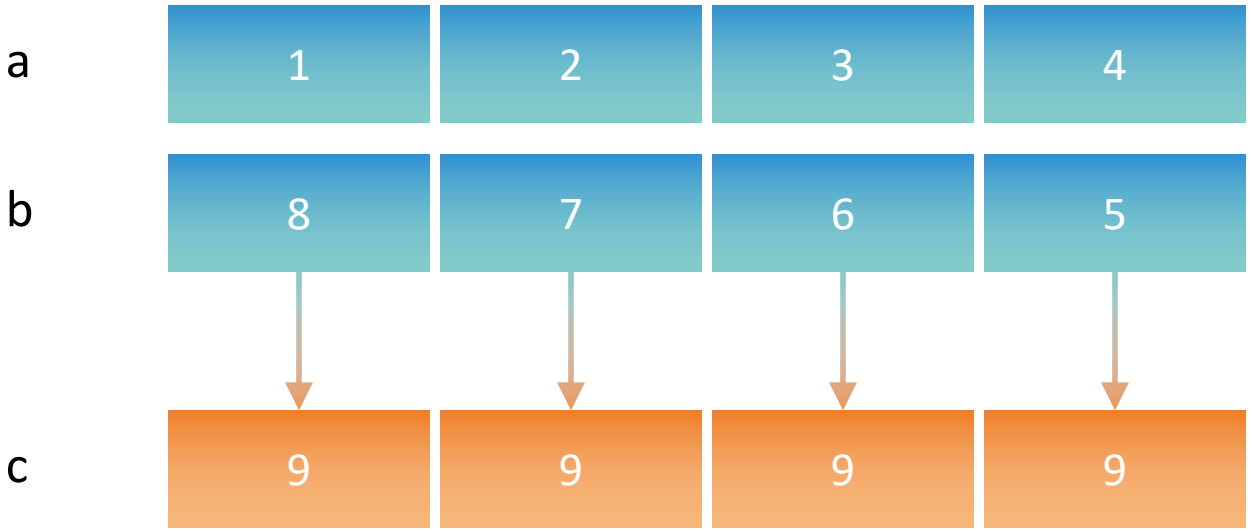
05 Matrix Multiplication

06 Parallel Reduction

Recap: Hardware Acceleration

上海创智学院
Shanghai Innovation Institute

The same four additions need work assignment and data movement




```
c[i] = a[i] + b[i];
```

Example: i = 2
Read 3 and 6
Add once; write 9

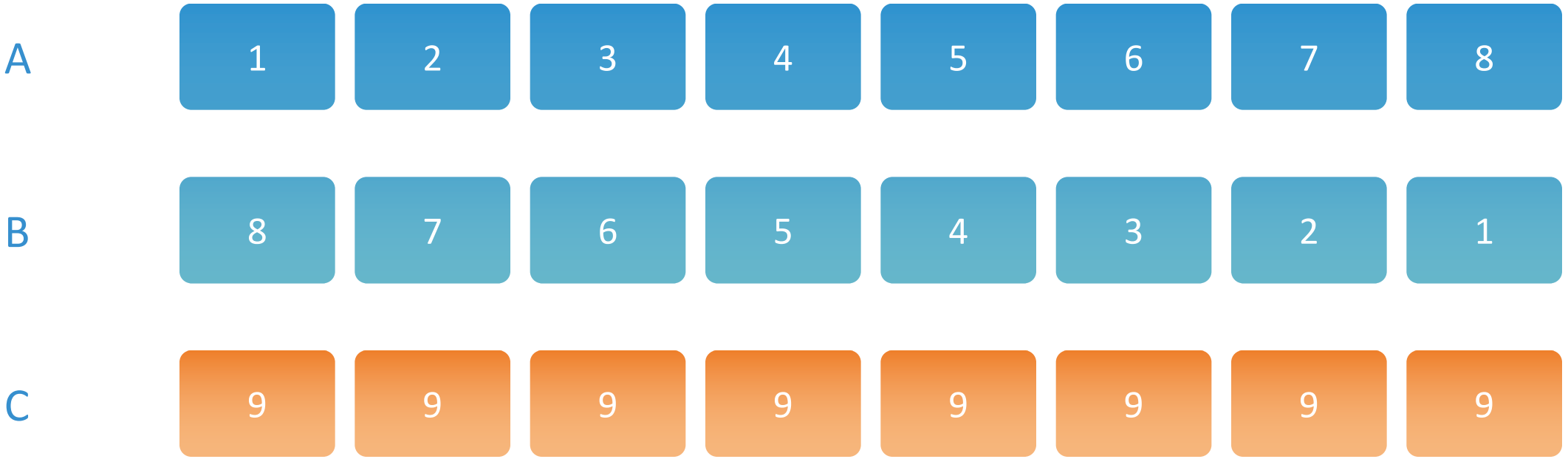
Each output is independent. The two input reads still have a cost.

01

GPU Architecture

Independent Work in an Array

上海创智学院
Shanghai Innovation Institute



Each output can be computed independently.

Latency and Throughput

上海创智学院
Shanghai Innovation Institute

16 independent tasks; each task takes 1 time unit

1 execution slot



16 time units

4 execution slots

0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

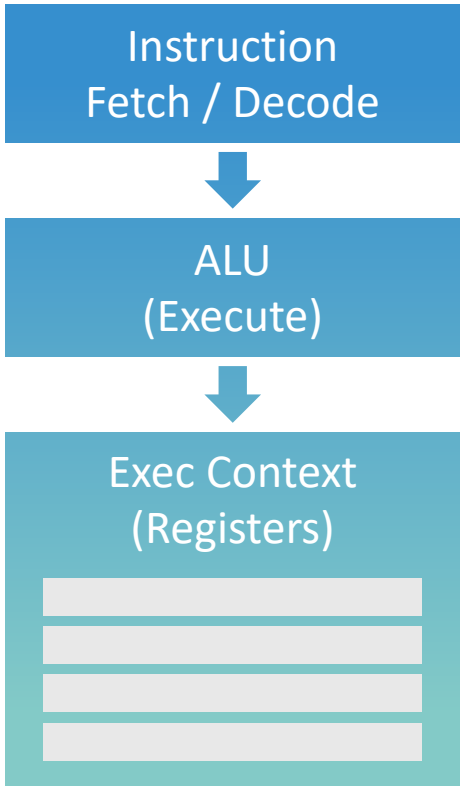
A dependent chain still waits:

4 columns = 4 time units



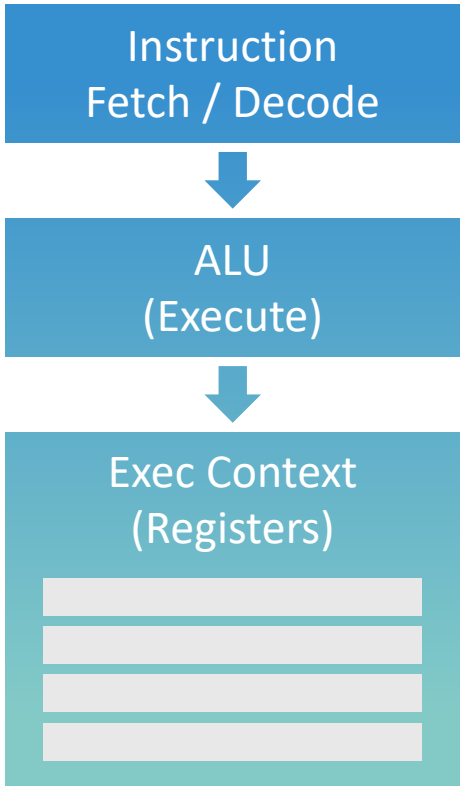
Teaching model, not GPU clock cycles. Parallel throughput does not shorten each task.

SISD: Single Instruction, Single Data

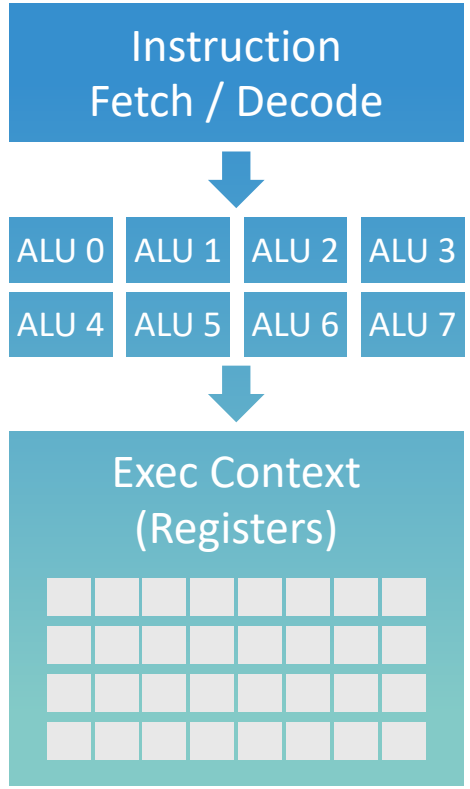


Conventional single instruction, single data processor

SIMD: Single Instruction, Multiple Data



Conventional single instruction, single data processor

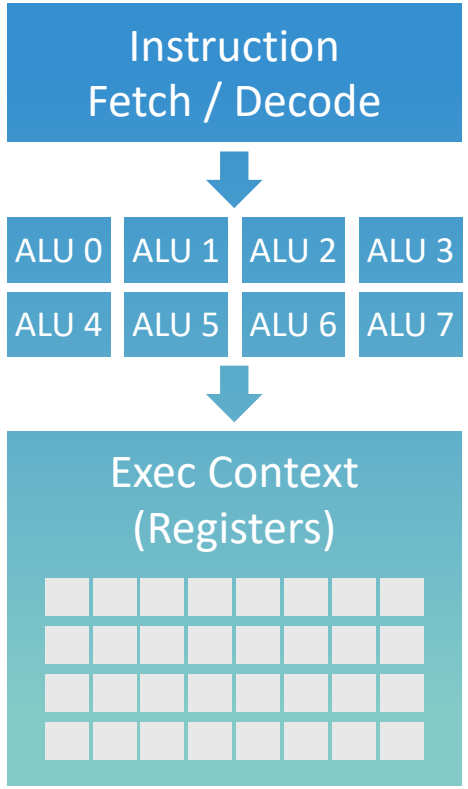


Modern single instruction, multiple data processor

SIMD: Single Instruction, Multiple Data

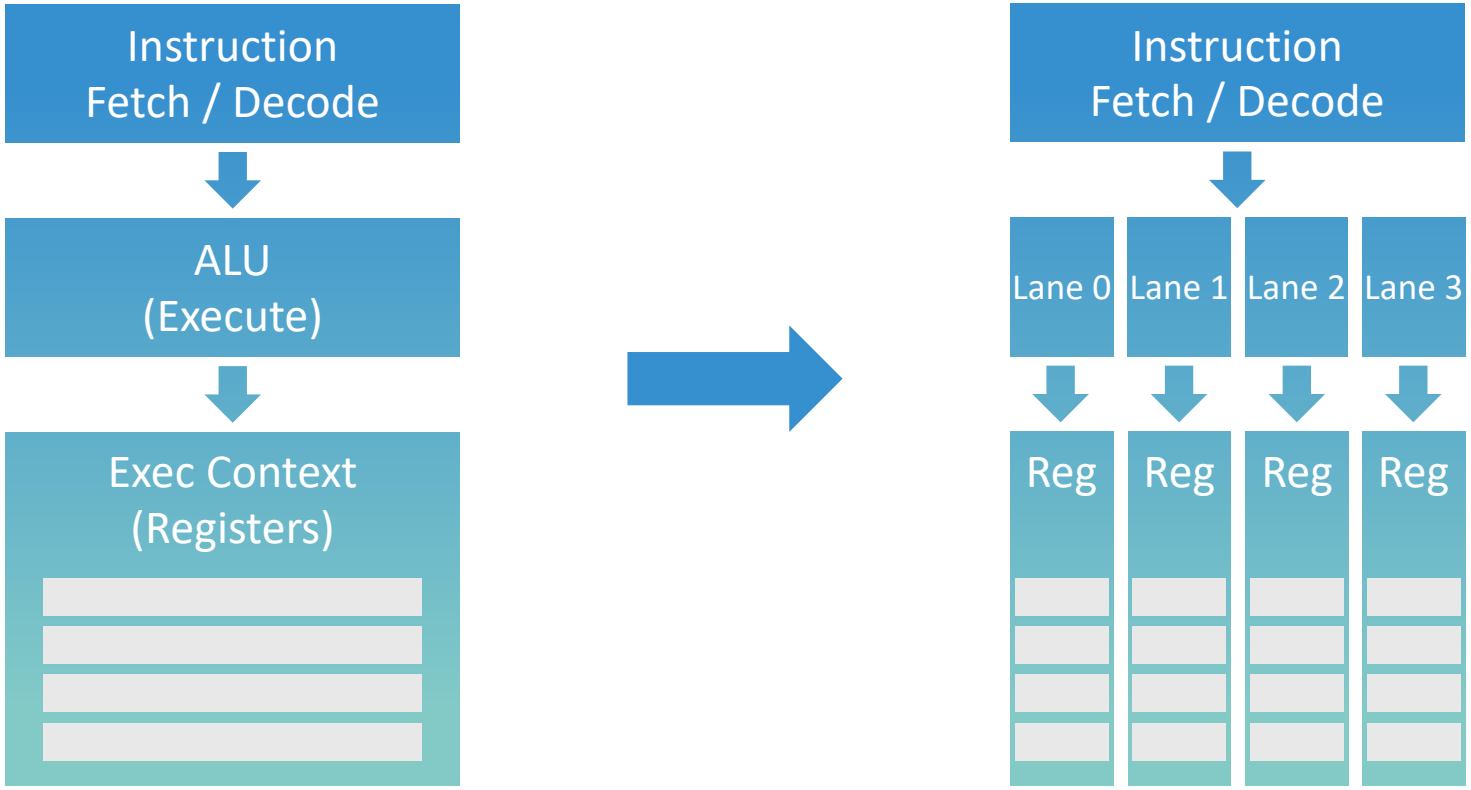
上海创智学院
Shanghai Innovation Institute

- Same instruction broadcast and executed in parallel on all ALUs
- Add ALUs to increase compute capability
- Used for vectorized execution



Modern single instruction, multiple data processor

SIMT: Single Instruction, Multiple Threads



Conventional single instruction, single data processor

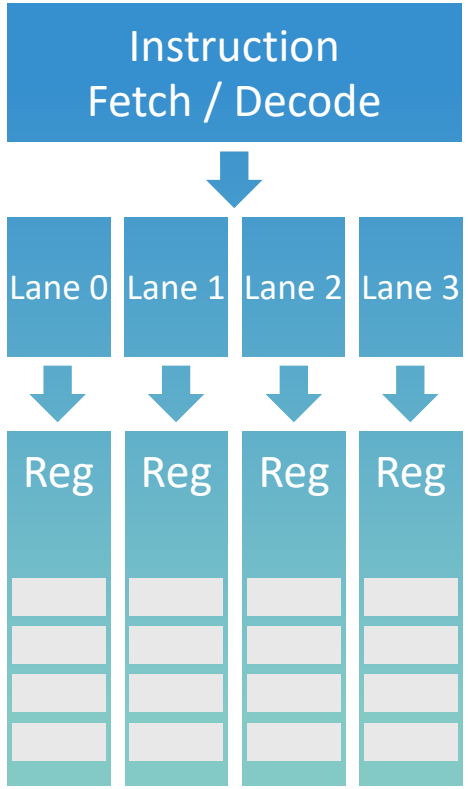
Modern single instruction, multiple thread GPUs

Logical lanes and per-thread state; physical registers reside in an SM register file.

SIMT: Single Instruction, Multiple Threads

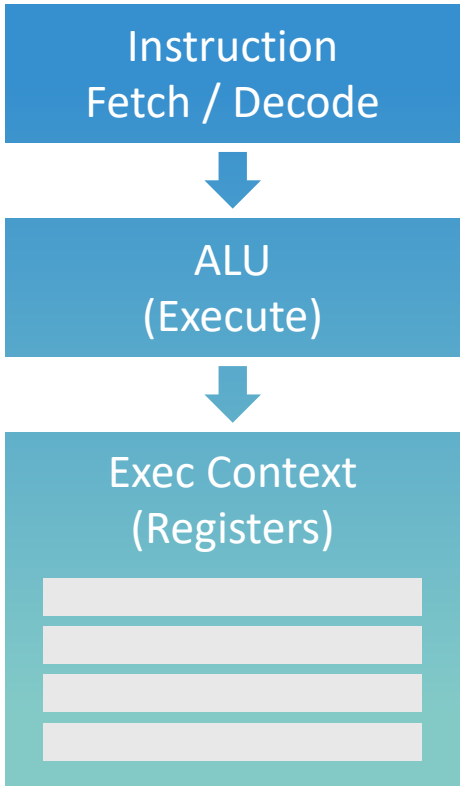
上海创智学院
Shanghai Innovation Institute

SIMT exposes scalar threads
Each thread has private register state
Threads share physical SM resources

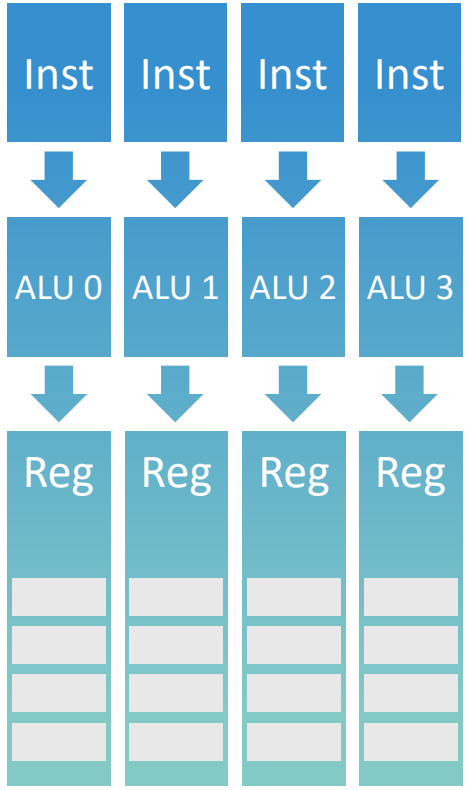


Logical SIMT view
4 of 32 lanes shown

MIMD: Multiple Instruction, Multiple Data





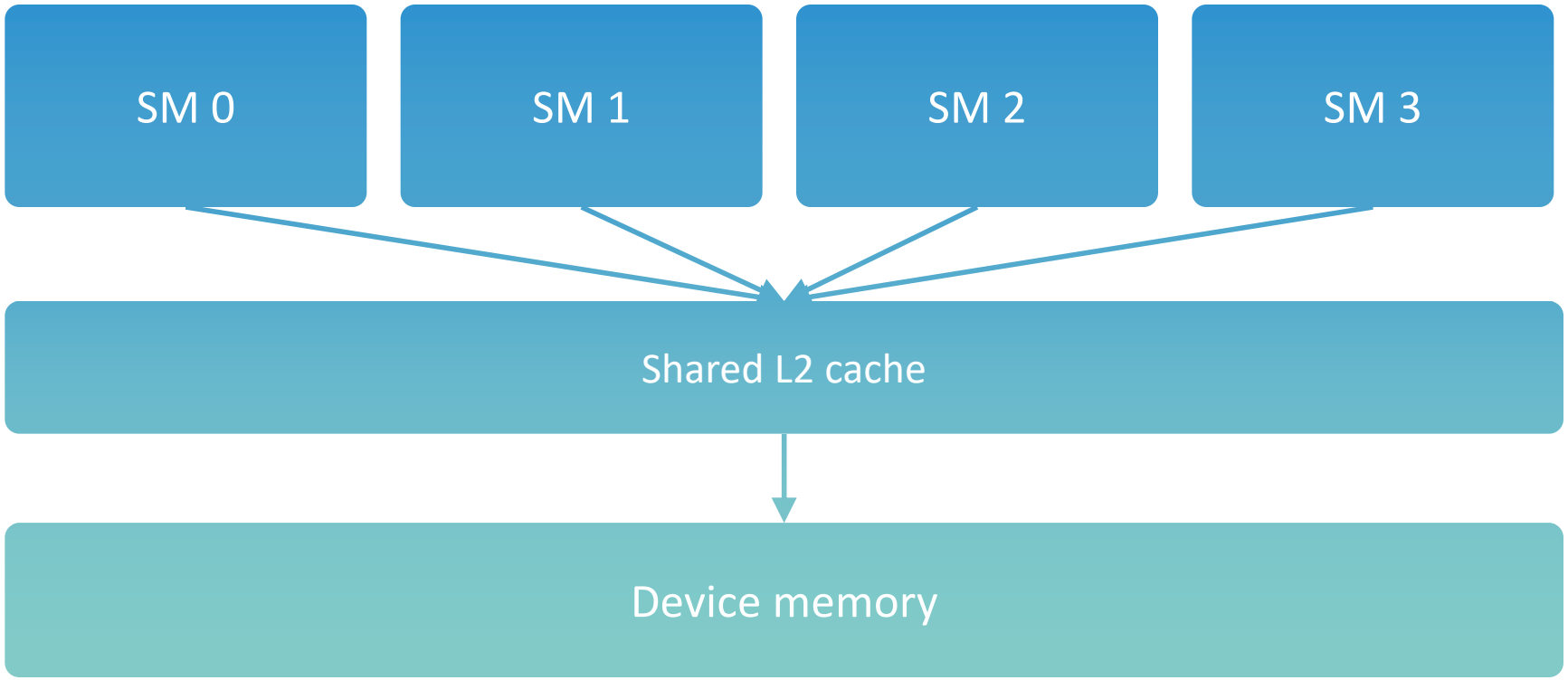


Conventional single instruction, single data processor

Modern multiple instruction, multiple data processor

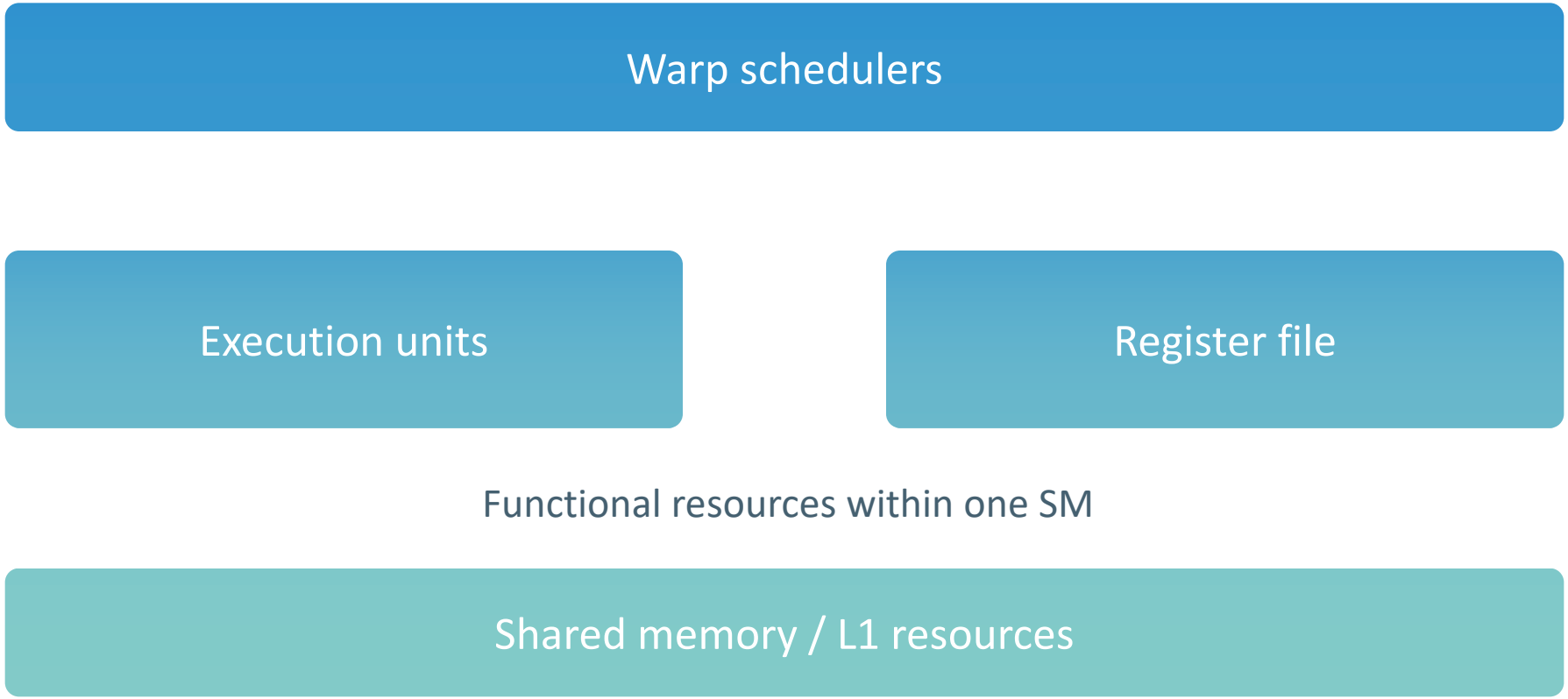
A Simplified GPU

上海创智学院
Shanghai Innovation Institute



Many SMs share access to device memory. Counts are illustrative.

Inside a Streaming Multiprocessor

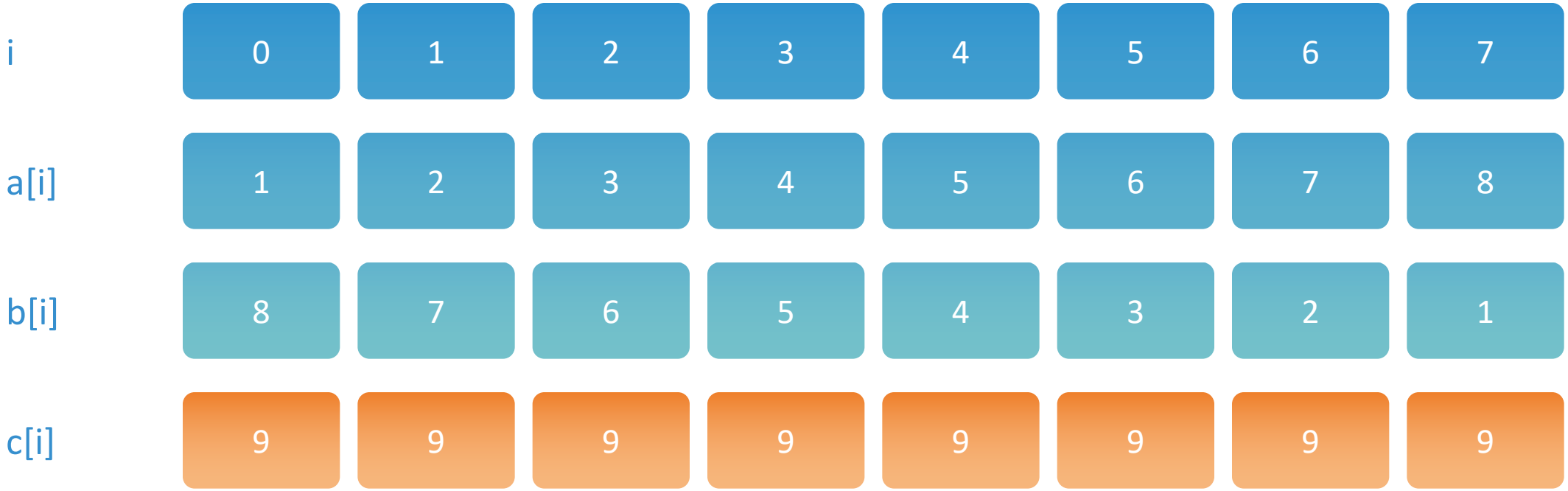


Execution and storage resources are finite and shared.

SIMT: One Program, Many Threads

上海创智学院
Shanghai Innovation Institute

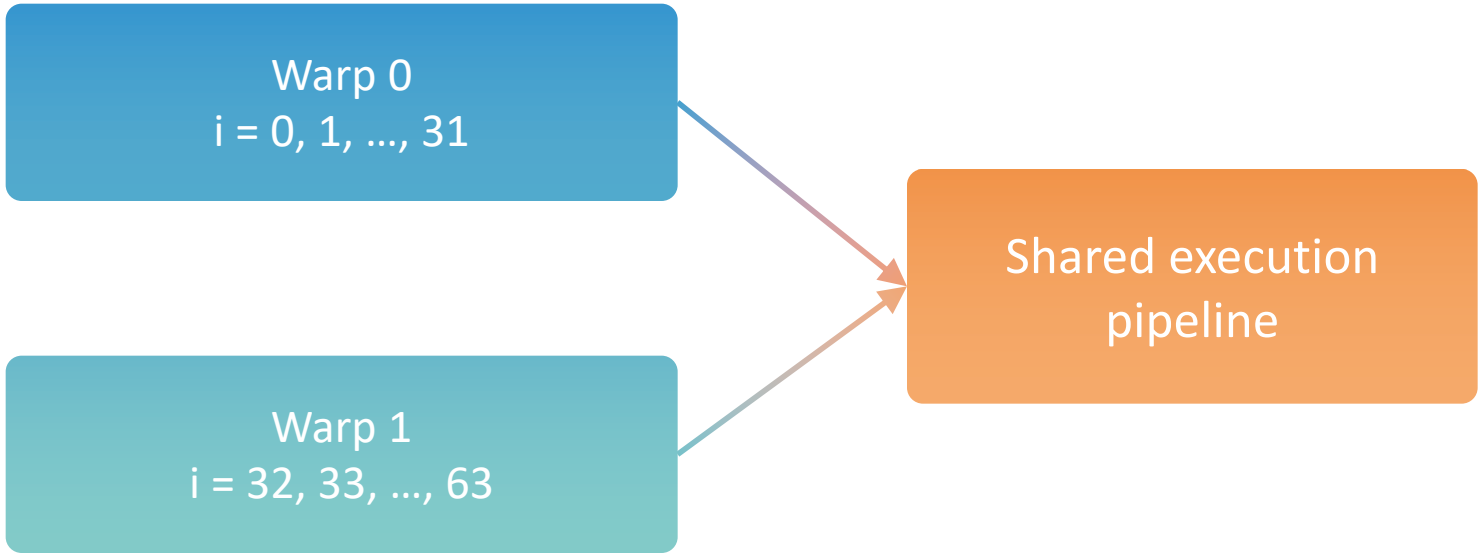
```
c[i] = a[i] + b[i]
```



A CUDA warp contains 32 threads. Eight lanes are shown.

Threads and Execution Units

One 64-thread block: two warps, with separate thread state



Issue opportunity A:
warp 0 performs an add

Issue opportunity B:
warp 1 uses the pipeline

Thread state stays allocated while the warp is resident.

A thread is not permanently paired with one CUDA core. Issue opportunities are not cycles.

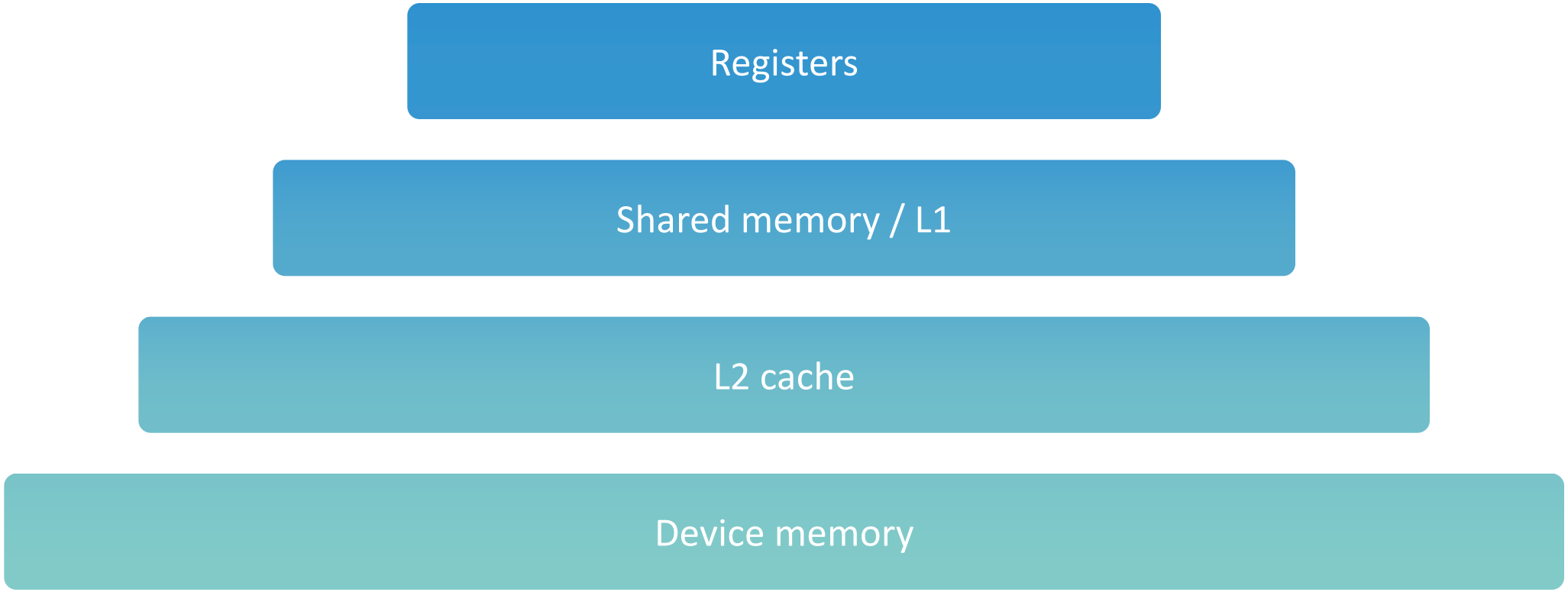
Hiding a Wait with Other Work



Illustrative issue order for one scheduler. Steps are not clock cycles.

Independent ready work can fill otherwise unused issue opportunities.

GPU Memory Hierarchy



Capacity, access cost and sharing scope differ across levels.

Registers Hold Thread State

上海创智学院
Shanghai Innovation Institute

```
int i = 2 * threadIdx.x;
float sum = 0;
sum += x[i];
sum += x[i + 1];
```

x = [1, 2, 3, 4]

Thread 0: i = 0



Thread 1: i = 2



The name sum is shared in the source code; its value is private to each thread.

Variables may be optimized away or spilled; private semantics do not guarantee registers.

Shared Memory Enables Cooperation

上海创智学院
Shanghai Innovation Institute

```
tile[t] = input[blockIdx.x * 4 + t];
__syncthreads();
```

Block 0: its own shared tile



tile[0] = 10

Block 1: its own shared tile



tile[0] = 50

Same array name, different allocations

A four-thread illustration. Ordinary blocks cannot access each other’s shared allocation.

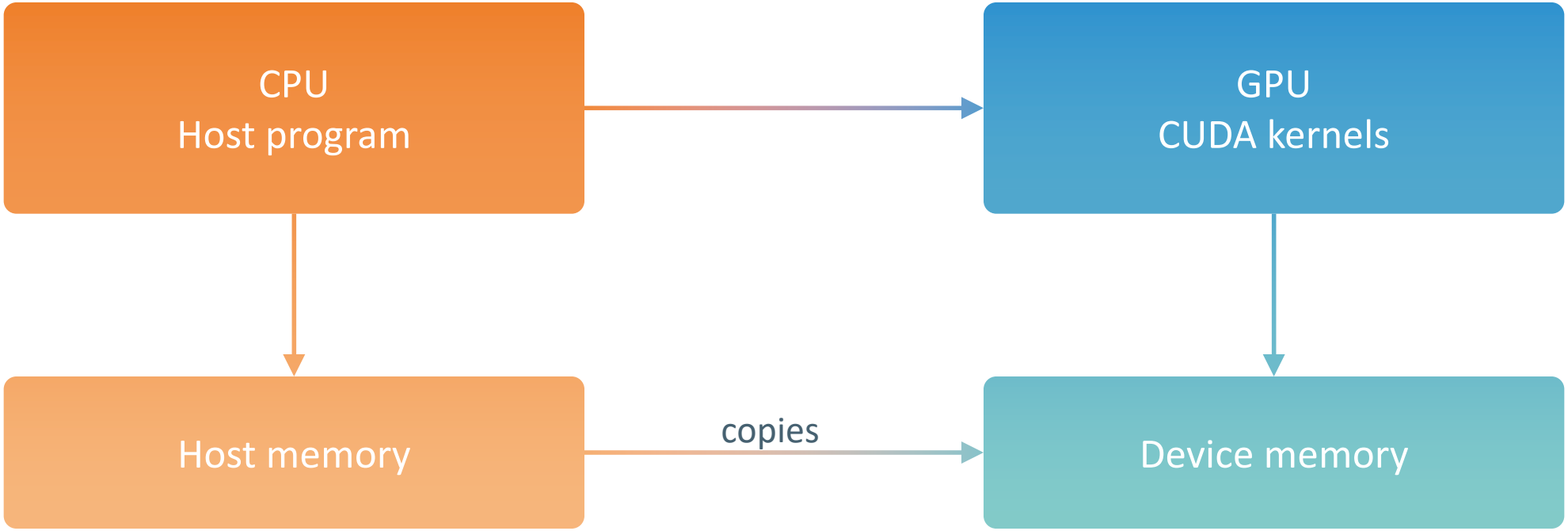
Memory Spaces and Scope

上海创智学院
Shanghai Innovation Institute

Space	Who can use it?	Managed by
Registers	One thread	Compiler / hardware
Shared memory	One block	Program
Global memory	All permitted threads	Program allocation
L1 / L2 cache	Requests using the cache	Hardware

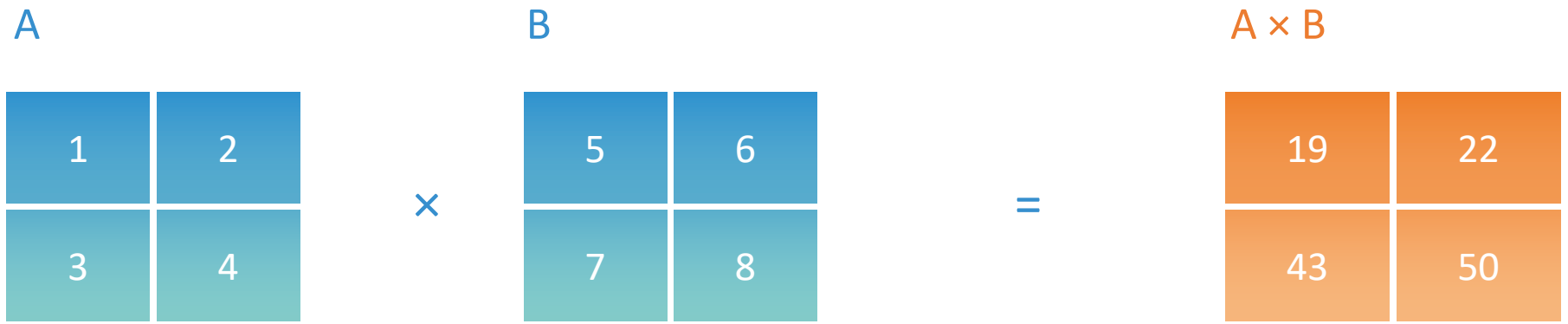
Scope does not replace synchronization or ownership rules.

CPU and GPU Work Together



Teaching setup: a discrete GPU with explicit host/device copies.

Scalar and Matrix Execution




```
sum = 1 * 5 + 2 * 7;  // 19
```

Matrix instructions operate on supported matrix tiles.

One scalar dot product for one output

The 2 × 2 example shows the math, not a Tensor Core instruction shape.

H100 Architecture with Tensor Cores



Full GH100: 144 SMs. H100 SXM5: 132 enabled SMs.

H100 Streaming Multiprocessor

上海创智学院
Shanghai Innovation Institute

Hopper example: Tensor Memory Accelerator (TMA)
Asynchronous tensor copies between global and shared memory

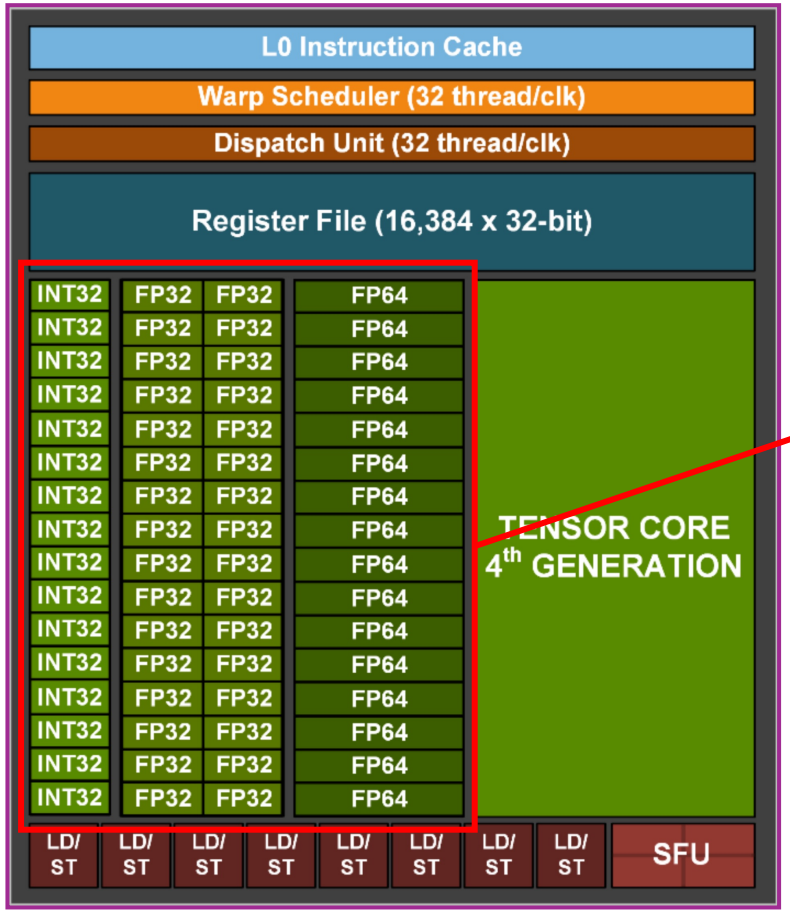
Shared Memory / L1 Data Cache



H100 Streaming Multiprocessor



H100 Streaming Multiprocessor

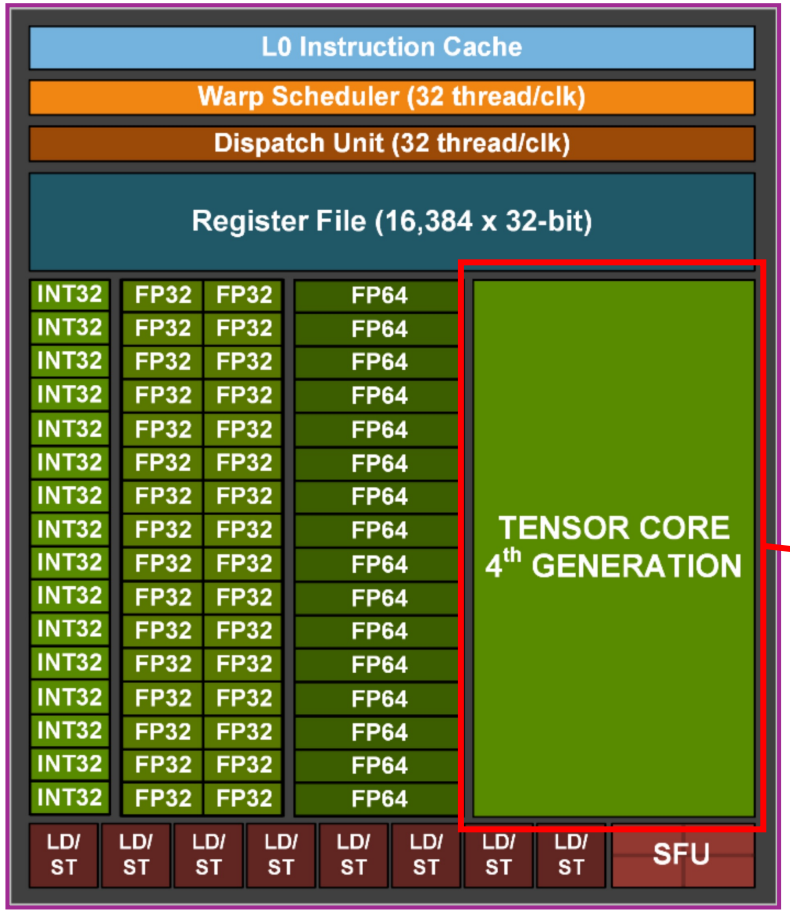


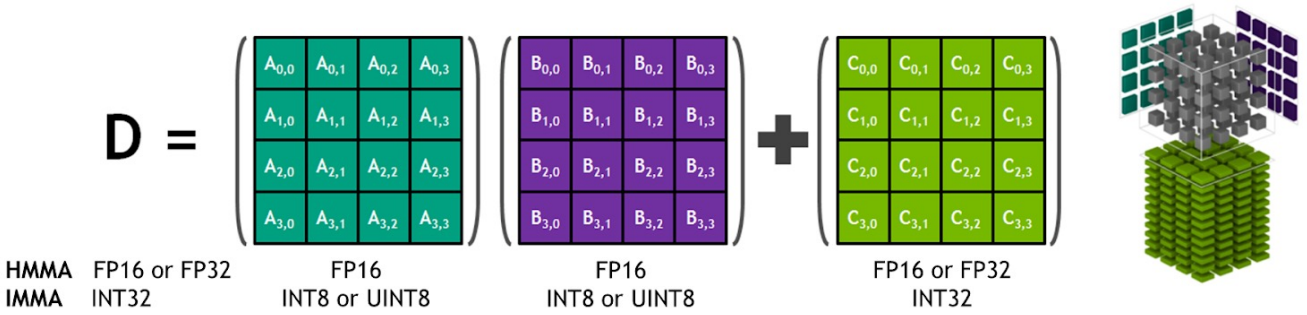
CUDA cores: scalar ALUs

Warp instructions use execution lanes

Threads share these units over time

H100 Streaming Multiprocessor





Tensor Cores, used for matrix multiplication

When GPU Parallelism Helps

上海创智学院
Shanghai Innovation Institute

Independent outputs

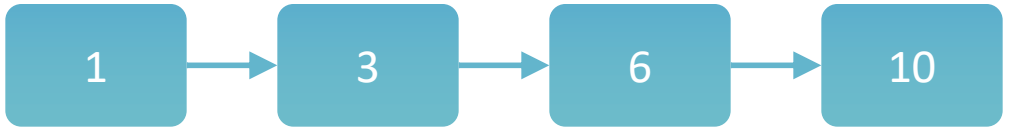
```
y[i] = 2 * x[i];
```



For x = [1, 2, 3, 4]
Each thread can work alone.

A dependent prefix

```
s[i] = s[i-1] + x[i];
```



Start with s[0] = x[0].
Each output needs the previous one.

A prefix sum needs a different parallel algorithm. Tiny arrays are illustrative, not benchmarks.

Discussion: Which Work Fits a GPU?

上海创智学院
Shanghai Innovation Institute

Compare three workloads.

1. A million independent vector elements
2. One short chain of dependent operations
3. A tiny kernel with large host/device transfers

Discuss parallelism, data movement and overhead before predicting speed.

02

CUDA Programming Abstractions

Basic CUDA Syntax

Host program: running as part of normal C/C++ application on CPU

Host

```
const int Nx = 12;
const int Ny = 6;
dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y, 1);

// A, B, C: device arrays [Ny][Nx]; exact tiling
// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each

matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Launch is asynchronous to the host.
Wait for completion before using results.

Device

```
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx]) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

`__global__` denotes a CUDA kernel runs on GPU

Each thread computes its overall grid thread id from its position in its block (threadIdx) and its block's position in the grid (blockIdx)

CUDA kernel: blocks execute on available SMs

Here Nx and Ny are divisible by the block dimensions.

Clear Separation of Host and Device Code

上海创智学院 Shanghai Innovation Institute

Host

```
const int Nx = 12;
const int Ny = 6;
dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y, 1);

// A, B, C: device arrays [Ny][Nx]; exact tiling
// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each

matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Device

```
__device__ float doubleValue(float x)
{
  return 2 * x;
}
__global__ void matrixAdd(float A[Ny][Nx],
                          float B[Ny][Nx],
                          float C[Ny][Nx]) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    C[j][i] = A[j][i] + B[j][i];
}
```

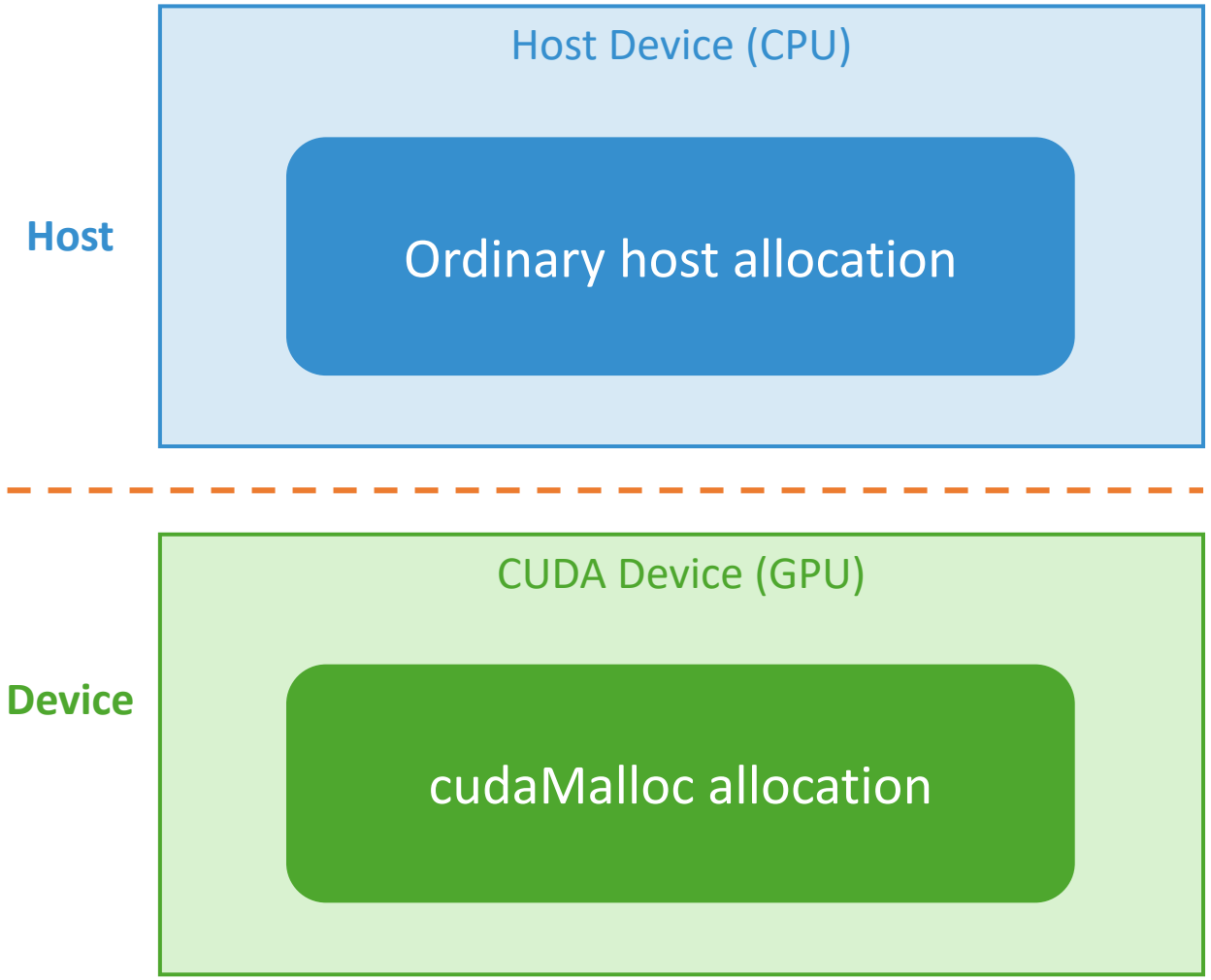
Separation of execution into host and device code is performed statically by the programmer

Function without any attribute runs on **Host** as common C++ program

__global__ denotes a CUDA kernel runs on GPU

__device__ denotes a CUDA function that can be called from **device** or **global** function

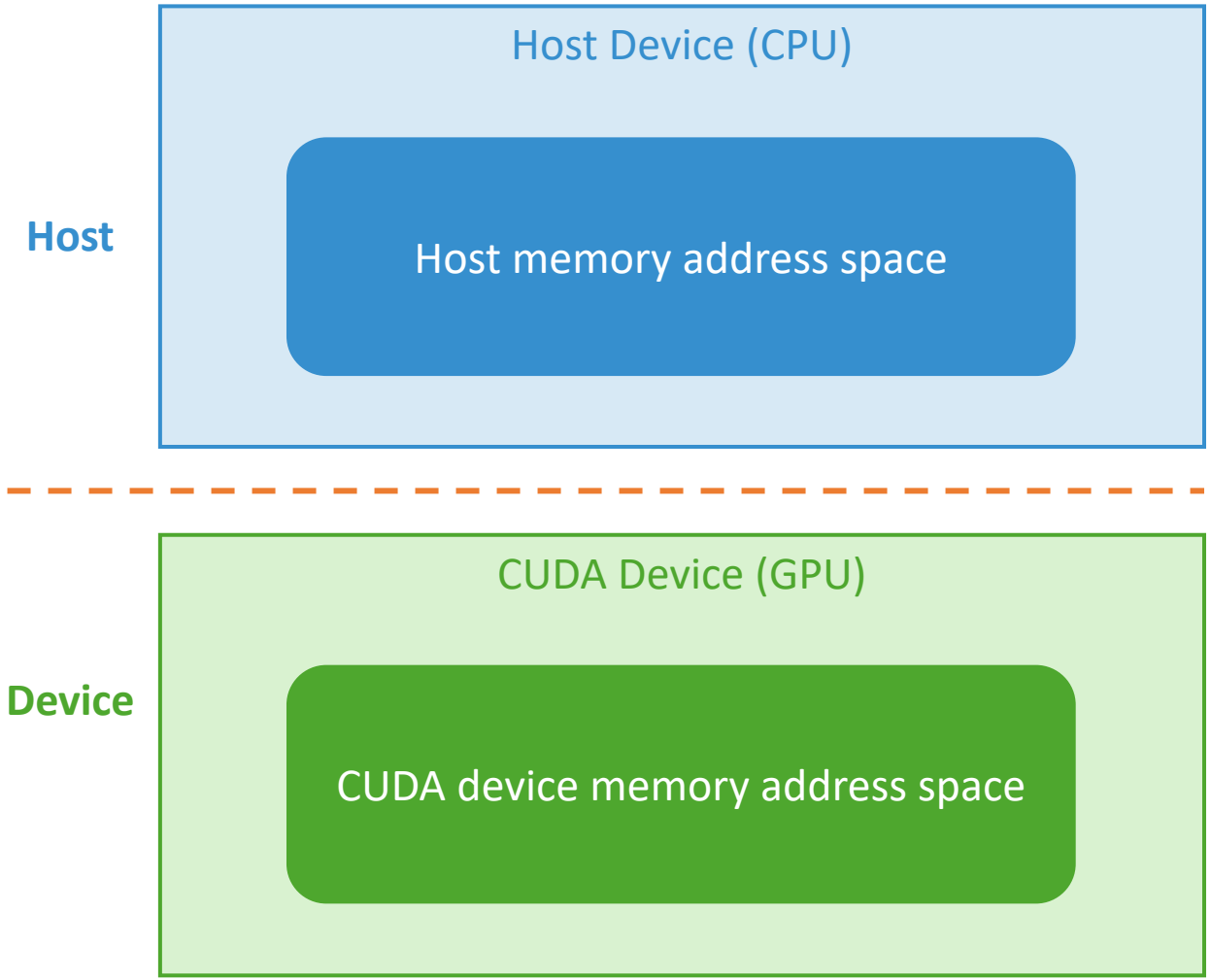
CUDA Memory Model



Conventional discrete-GPU example:
CPU uses ordinary host allocations.
GPU uses cudaMalloc allocations.
Copy data explicitly between them.

Unified Memory and mapped host memory are separate cases.

Case Study: Minimal Example




```
float* A = new float[N];

for (int i=0; i<N; i++)
  A[i] = (float)i;

size_t bytes = sizeof(float) * N;
float* deviceA;
cudaMalloc(&deviceA, bytes);

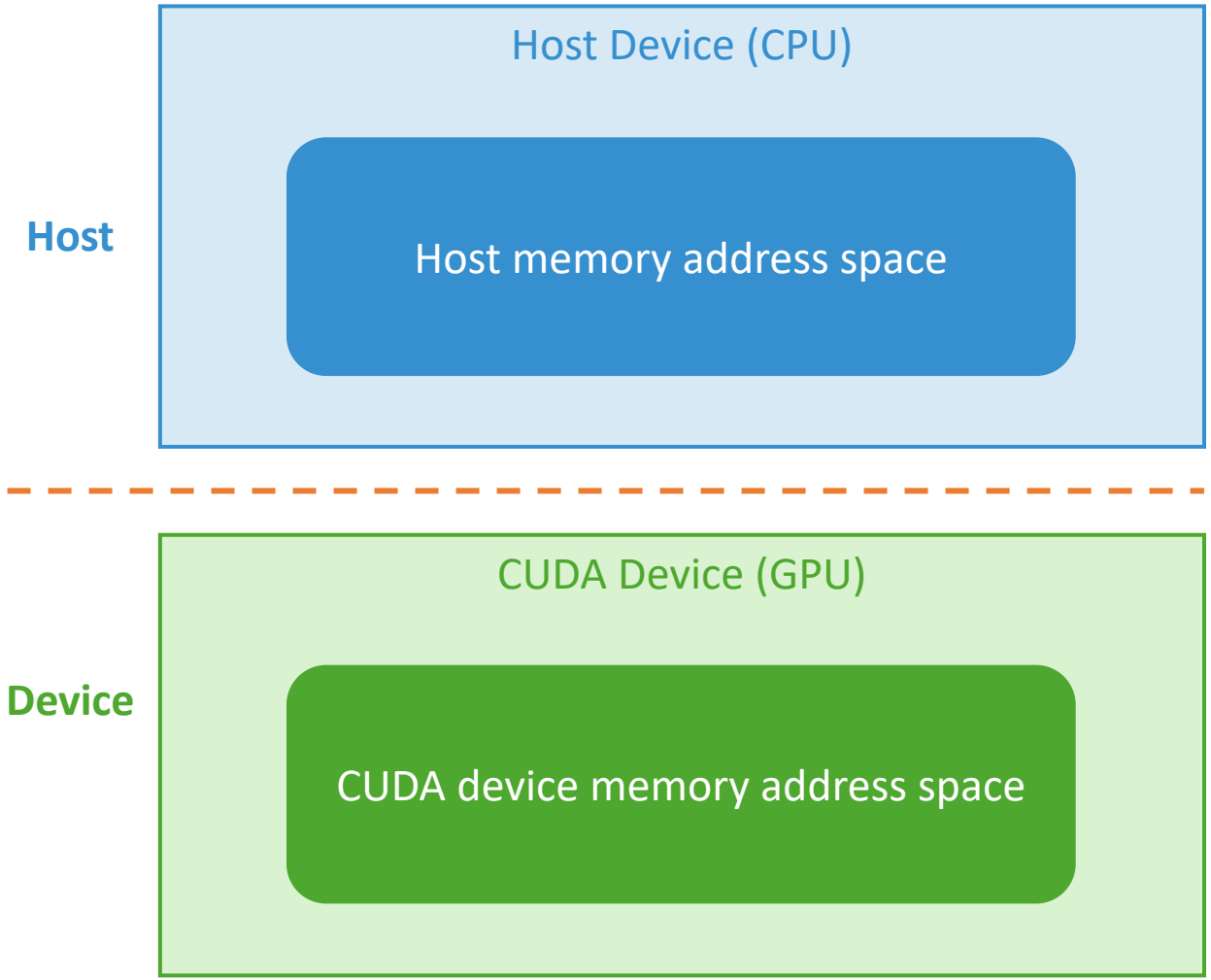
cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);
```

Init data on host device

Excerpt only. Check API errors and free allocations after use.

cudaMalloc: Allocate Memory Space on Device

上海创智学院 Shanghai Innovation Institute




```
float* A = new float[N];

for (int i=0; i<N; i++)
  A[i] = (float)i;

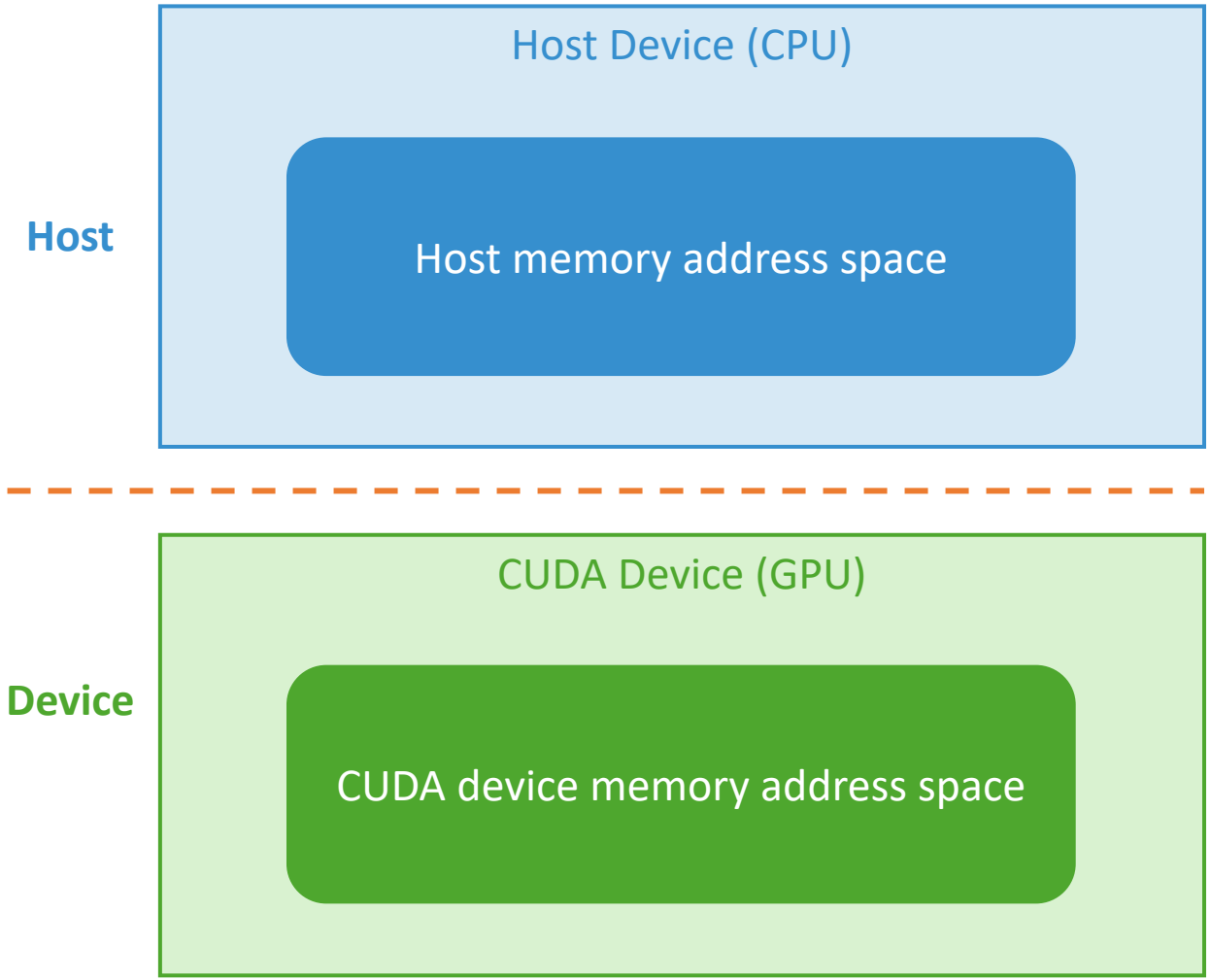
size_t bytes = sizeof(float) * N;
float* deviceA;
cudaMalloc(&deviceA, bytes);

cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);
```

Allocate memory with specific size on device, and store the pointer to `deviceA`

Excerpt only. Check API errors and free allocations after use.

cudaMemcpy: Move Data Between Host and Device




```
float* A = new float[N];

for (int i=0; i<N; i++)
  A[i] = (float)i;

size_t bytes = sizeof(float) * N;
float* deviceA;
cudaMalloc(&deviceA, bytes);

cudaMemcpy(deviceA, A, bytes, cudaMemcpyHostToDevice);
```

Copy data from host to device

Here deviceA comes from cudaMalloc:
CPU code cannot dereference deviceA.
cudaMemcpy also supports device-to-host copies.

Excerpt only. Check API errors and free allocations after use.

Minimal Example: Launch and Read Back

上海创智学院
Shanghai Innovation Institute

After allocation + H2D: deviceA = [0, 1, 2, 3]

Device: one function, four threads

```
__global__ void scale(float* a, int n) {
  int i = blockIdx.x * blockDim.x
        + threadIdx.x;
  if (i < n) a[i] *= 2;
}
```

Host: submit, wait, copy back

```
scale<<<1, 4>>>(deviceA, 4);
CHECK(cudaGetLastError());
CHECK(cudaDeviceSynchronize());
CHECK(cudaMemcpy(A, deviceA,
  4 * sizeof(float),
  cudaMemcpyDeviceToHost));
```

T2 reads 2 and writes 4.

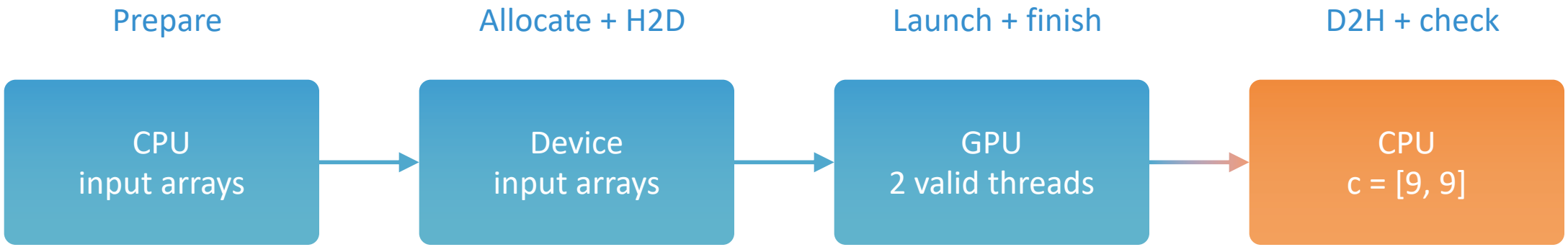
CPU result: [0, 2, 4, 6]

<<<1, 4>>> = 1 block × 4 threads

N = 4; launch size is for illustration. Then cudaFree(deviceA) and delete[] A. CHECK is defined later.

A Complete CUDA Program

CPU inputs: a = [1, 2], b = [8, 7]



T0: 1 + 8 → 9
T1: 2 + 7 → 9

Copying pointers alone does not copy their arrays.

Discrete GPU, explicit copies, one stream. Confirm completion, validate, then free allocations.

Vector Addition Kernel

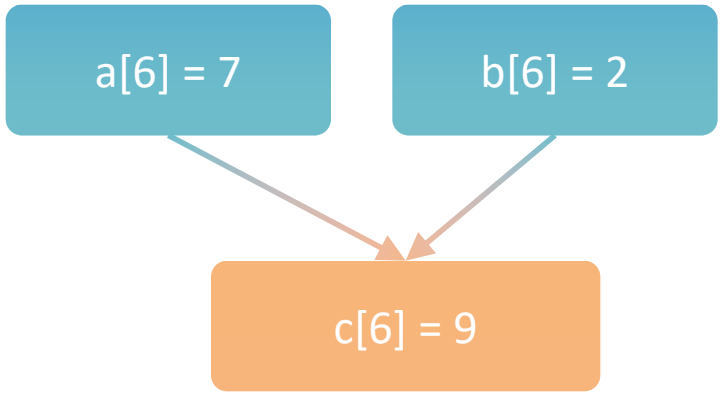
上海创智学院
Shanghai Innovation Institute

Trace one thread: block = 1, block size = 4, local thread = 2

```
__global__ void vec_add(
  const float *a, const float *b,
  float *c, int n) {
  int i=blockIdx.x*blockDim.x+threadIdx.x;
  if (i<n) c[i]=a[i]+b[i];
}
```

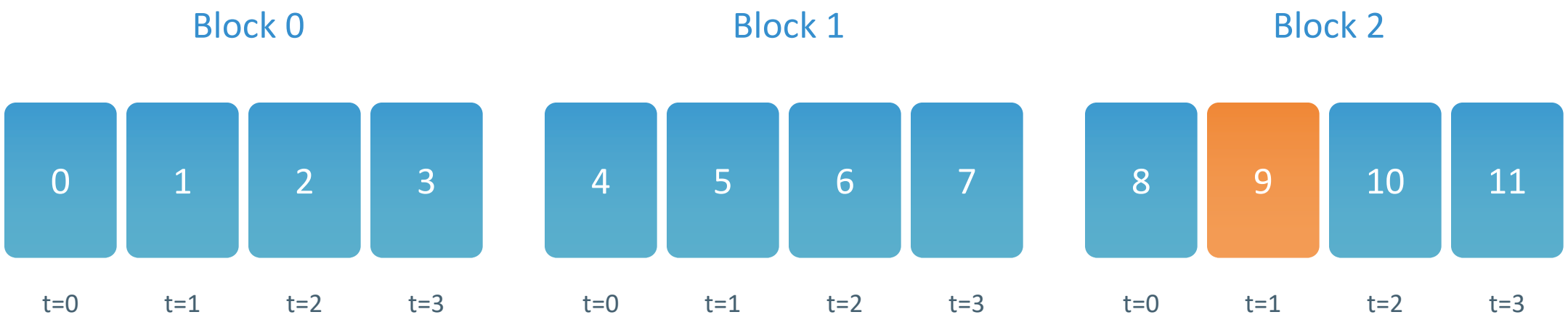
$i = 1 \times 4 + 2 = 6$

n = 10: the guard passes



Per valid thread: read 2 FP32 values (8 B), write 1 value (4 B). Logical requests, not HBM transactions.

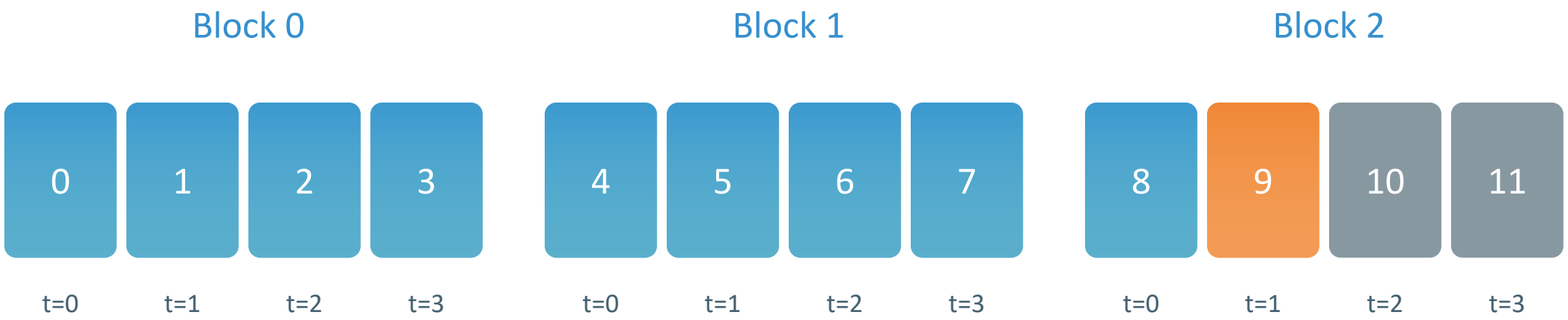
Computing a Global Index



Block 2, thread 1: 2 × 4 + 1 = 9

i = blockIdx.x * blockDim.x + threadIdx.x

The Final Partial Block

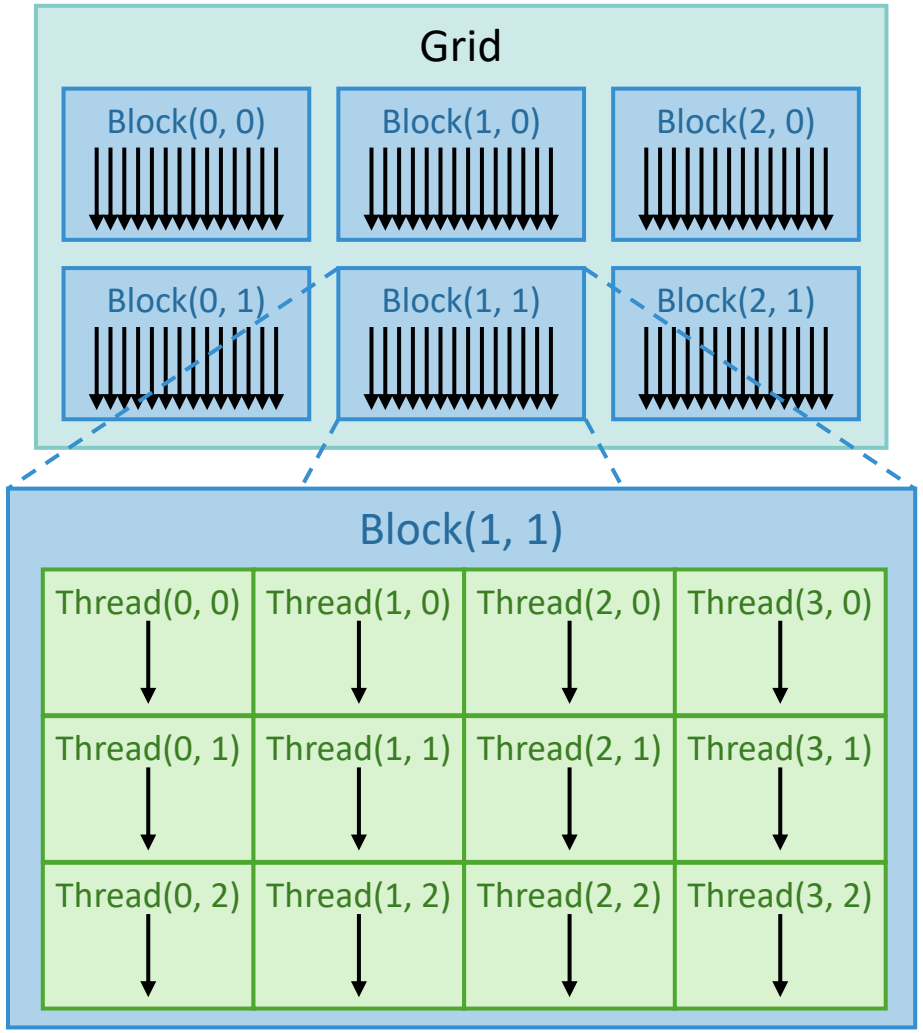


Valid indices: 0 ... 9

n = 10, block size = 4: launch 12 threads and guard i < n.

CUDA Programs Consist of a Hierarchy of Threads

`threadIdx` and `blockIdx` are up to 3-dimensional (2D example below)



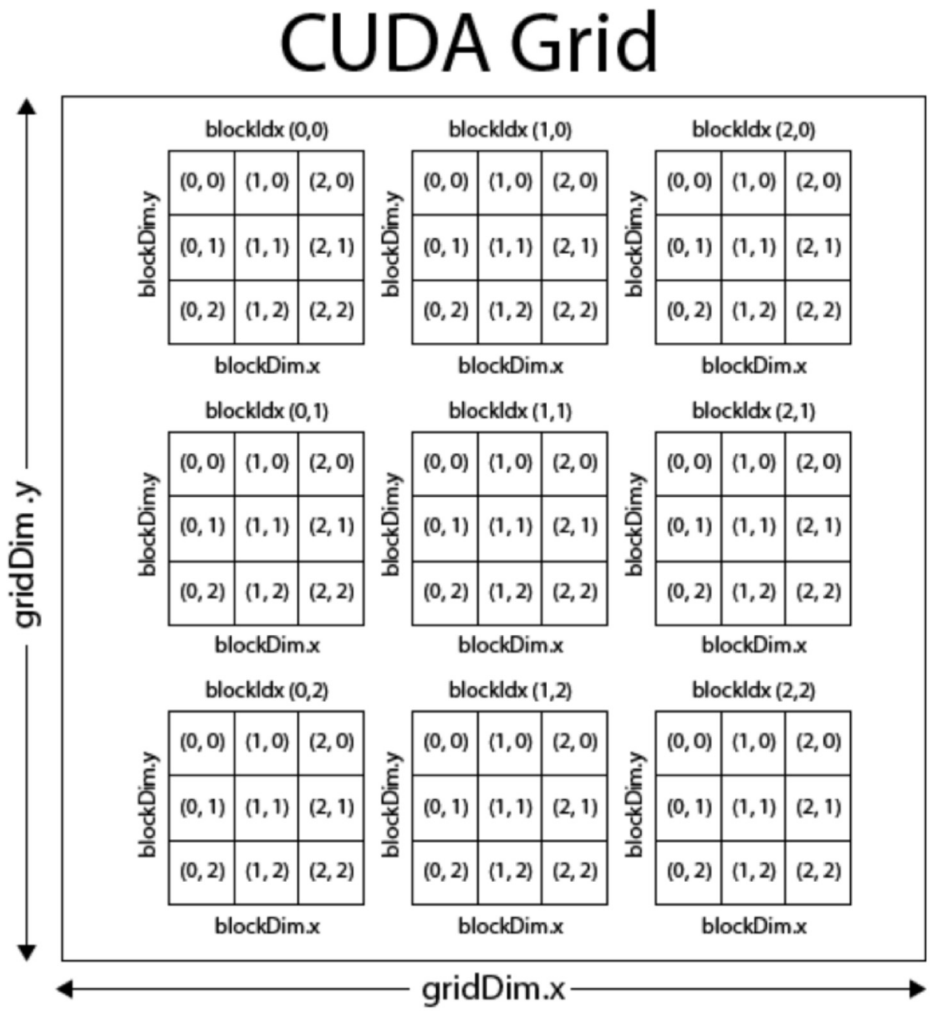

```
const int Nx = 12;
const int Ny = 6;
dim3 threadsPerBlock(4, 3, 1);
dim3 numBlocks(Nx/threadsPerBlock.x, Ny/threadsPerBlock.y, 1);

// assume A, B, C are allocated Nx x Ny float arrays
// this call will trigger execution of 72 CUDA threads:
// 6 thread blocks of 12 threads each

matrixAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
```

Grid, Block, and Thread

- gridDim: The dimensions of the grid
- blockIdx: The block index within the grid
- blockDim: The dimensions of a block
- threadIdx: The thread index within a block



Matrix Addition

上海创智学院
Shanghai Innovation Institute

3 × 4 matrix; block = (2,2), blockIdx = (1,0), threadIdx = (0,1)

```
int col=blockIdx.x*blockDim.x+threadIdx.x;
int row=blockIdx.y*blockDim.y+threadIdx.y;
if (row<rows && col<cols) {
  int i=row*cols+col;
  c[i]=a[i]+b[i];
}
```

row = 1, col = 2
i = 1 × 4 + 2 = 6

0	1	2	3
4	5	6	7
8	9	10	11

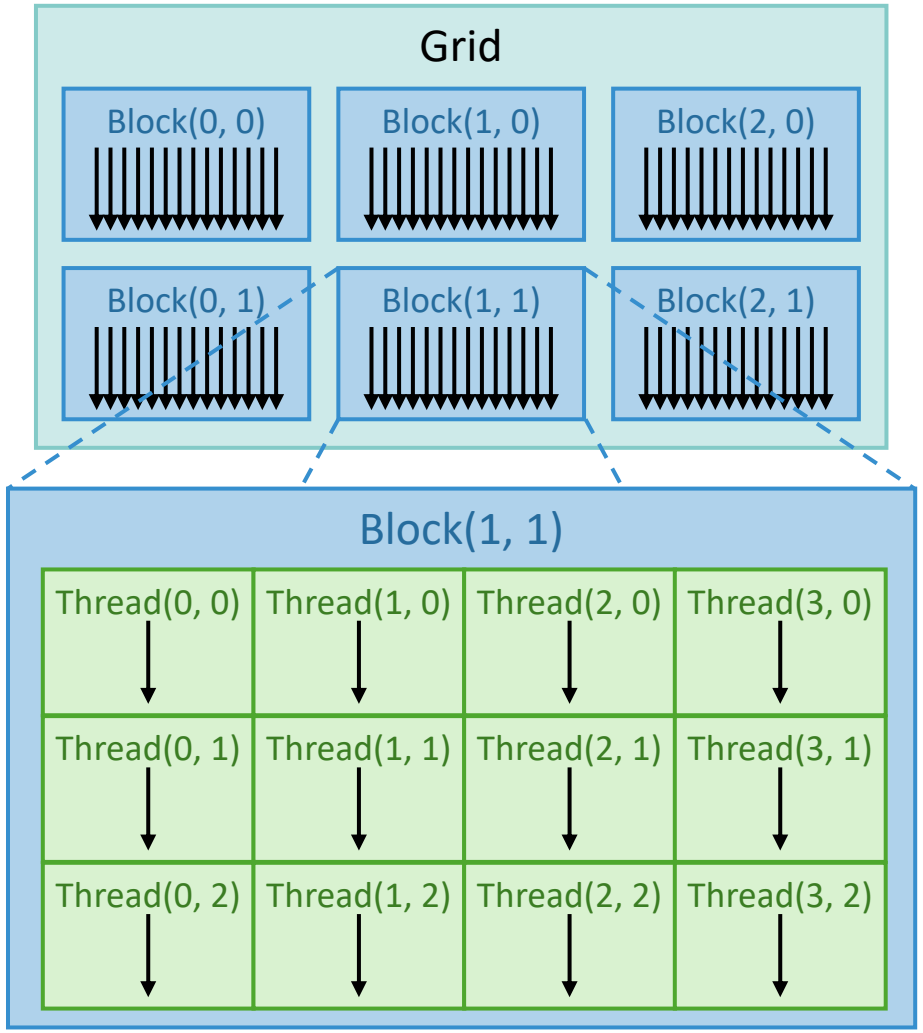
a[6] = 7, b[6] = 2
c[6] = 9

Core CUDA code. Matrix coordinates and linear addresses describe the same output element.

CUDA Blocks Map to GPU SM

上海创智学院
Shanghai Innovation Institute

A block executes on one SM. The grid need not occupy every SM.

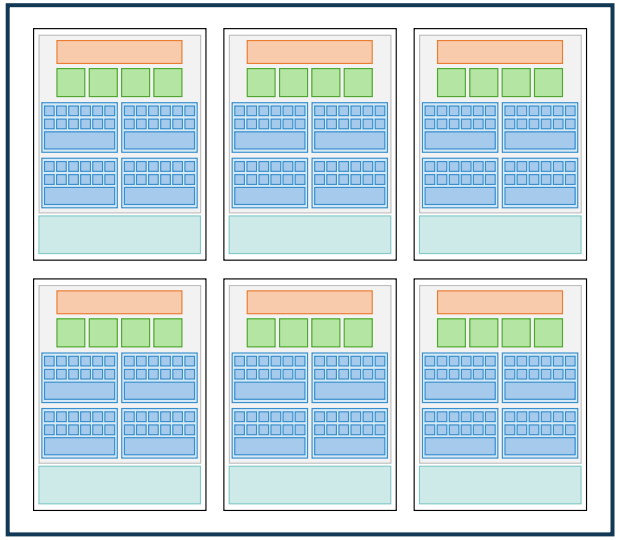






Scalable Thread Block Execution

Independent blocks can be distributed across different numbers of SMs.



Example GPU (6 SMs)



Example GPU (12 SMs)

CUDA Thread Block Scheduling

上海创智学院
Shanghai Innovation Institute

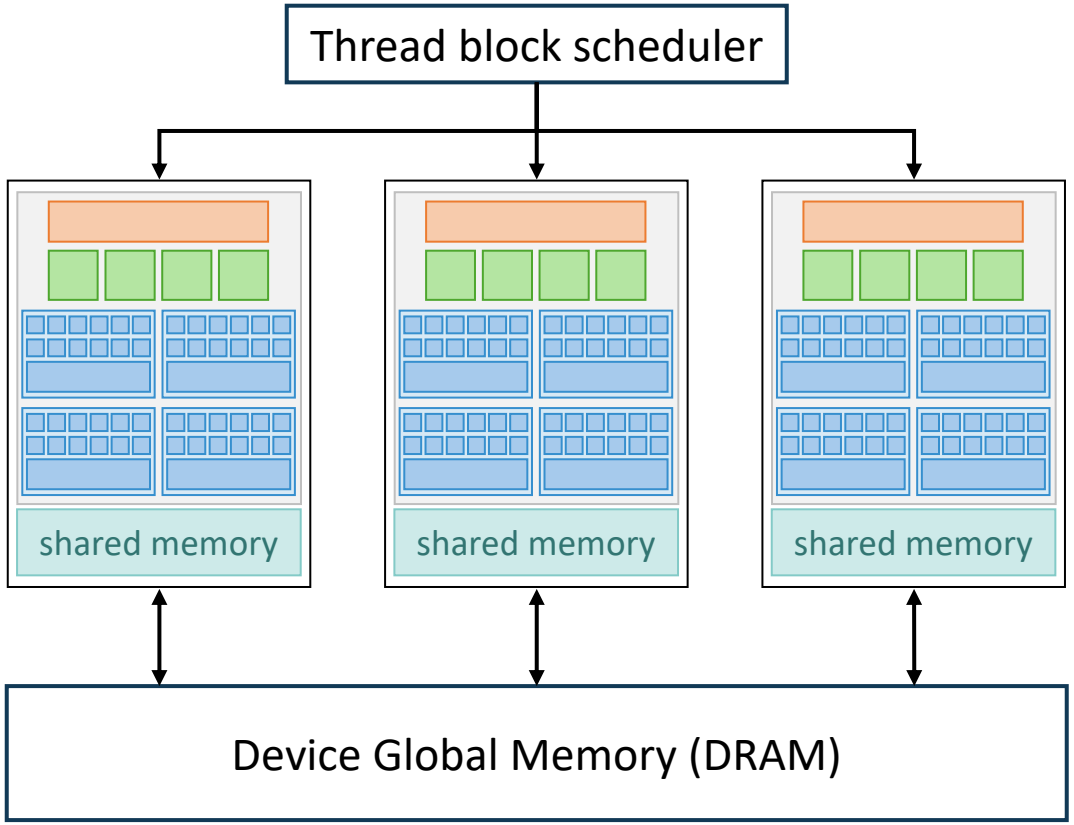
Ordinary CUDA kernels:

Blocks must work in any scheduling order.

The GPU assigns blocks to SMs when resources are available.

Several blocks may reside on one SM.

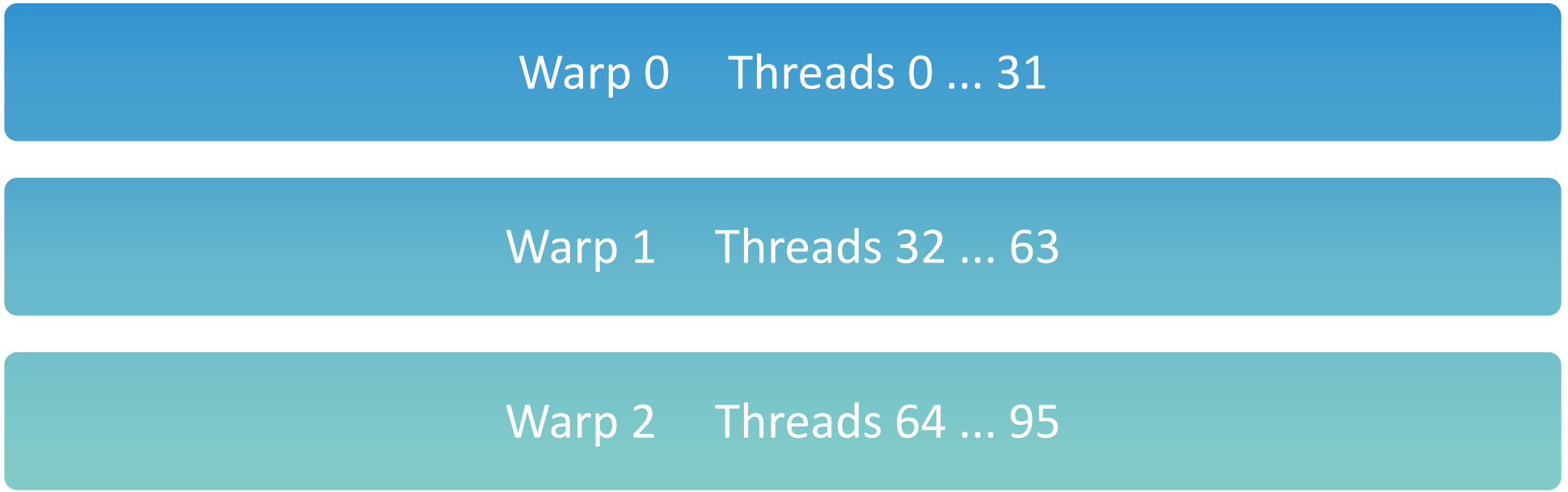
All SMs need not be active.



CUDA Warps

上海创智学院
Shanghai Innovation Institute

A block is partitioned into warps of 32 consecutive thread IDs.



One warp issues an instruction for the lanes active on that path.

Within a block, x varies fastest. A final warp may have fewer than 32 valid threads.

Warps Follow Linear Thread IDs

上海创智学院
Shanghai Innovation Institute

blockDim = (16, 4)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63

Warp 0: IDs 0–31

Warp 1: IDs 32–63

For blockDim = (16, 4), each warp spans two rows of threads.

Warp Scheduling: Instruction Issue

上海创智学院 Shanghai Innovation Institute

A scheduler selects a ready warp instruction.



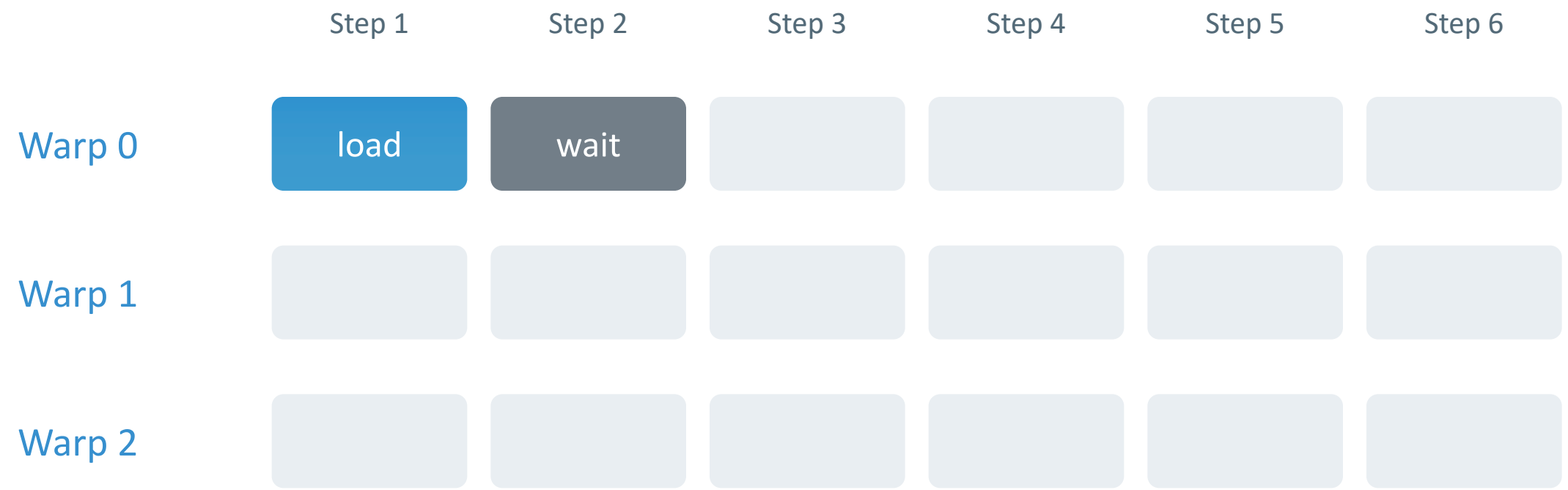
Warp 0 issues a load. The warp is still resident.

Illustrative issue sequence for one scheduler. Steps are not clock cycles or instruction latencies.

Warp Scheduling: Waiting for Data

上海创智学院
Shanghai Innovation Institute

A dependent instruction waits for its operands.



Warp 0 cannot yet issue the add that uses the loaded value.

Illustrative issue sequence for one scheduler. Steps are not clock cycles or instruction latencies.

Warp Scheduling: Hiding Latency

上海创智学院
Shanghai Innovation Institute

Other resident warps can issue while one warp waits.



Ready instructions from warp 1 and warp 2 fill issue opportunities.

Illustrative issue sequence for one scheduler. Steps are not clock cycles or instruction latencies.

Warp Scheduling: Resuming a Warp

上海创智学院 Shanghai Innovation Institute

A waiting warp becomes eligible when its dependency is resolved.



Warp 0 can issue its dependent add. Other warps need not have finished.

Illustrative issue sequence for one scheduler. Steps are not clock cycles or instruction latencies.

Resources Limit Residency

上海创智学院
Shanghai Innovation Institute

Hypothetical SM: 48 KiB available shared memory



A third 20 KiB allocation does not fit.

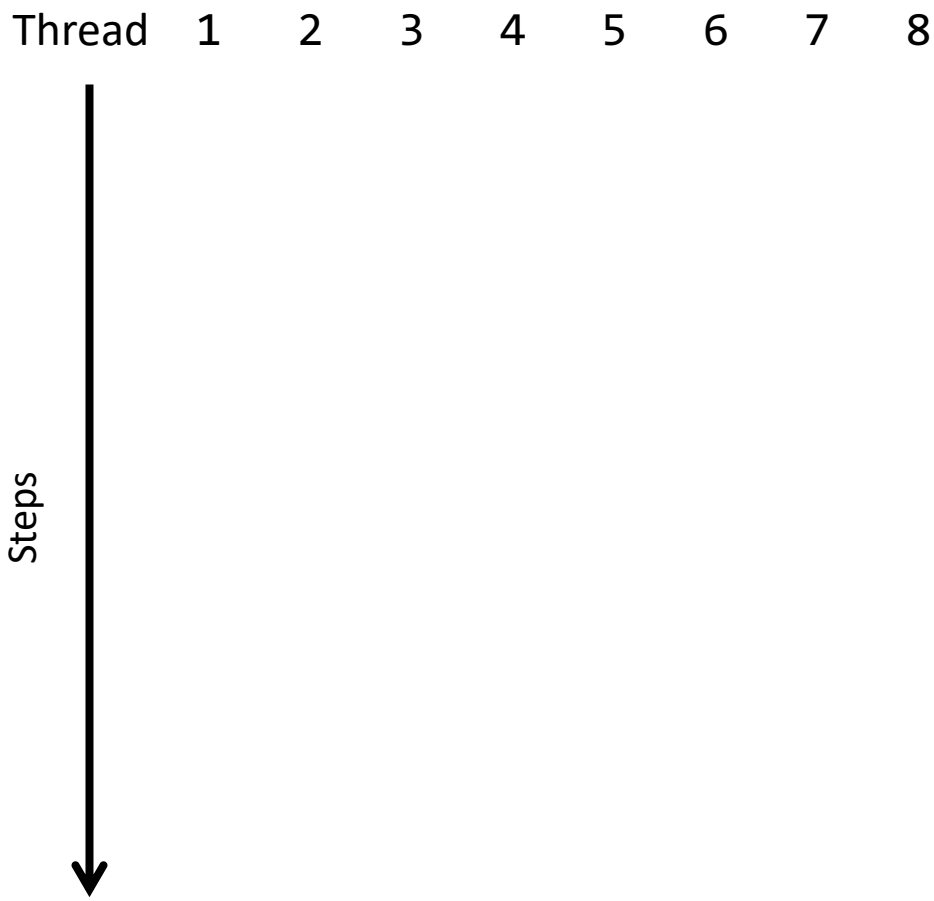
$$2 \times 256 = 512$$

threads = 16 resident warps

Each block has 256 threads. Assume other limits are not tighter; allocation is per whole block.

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



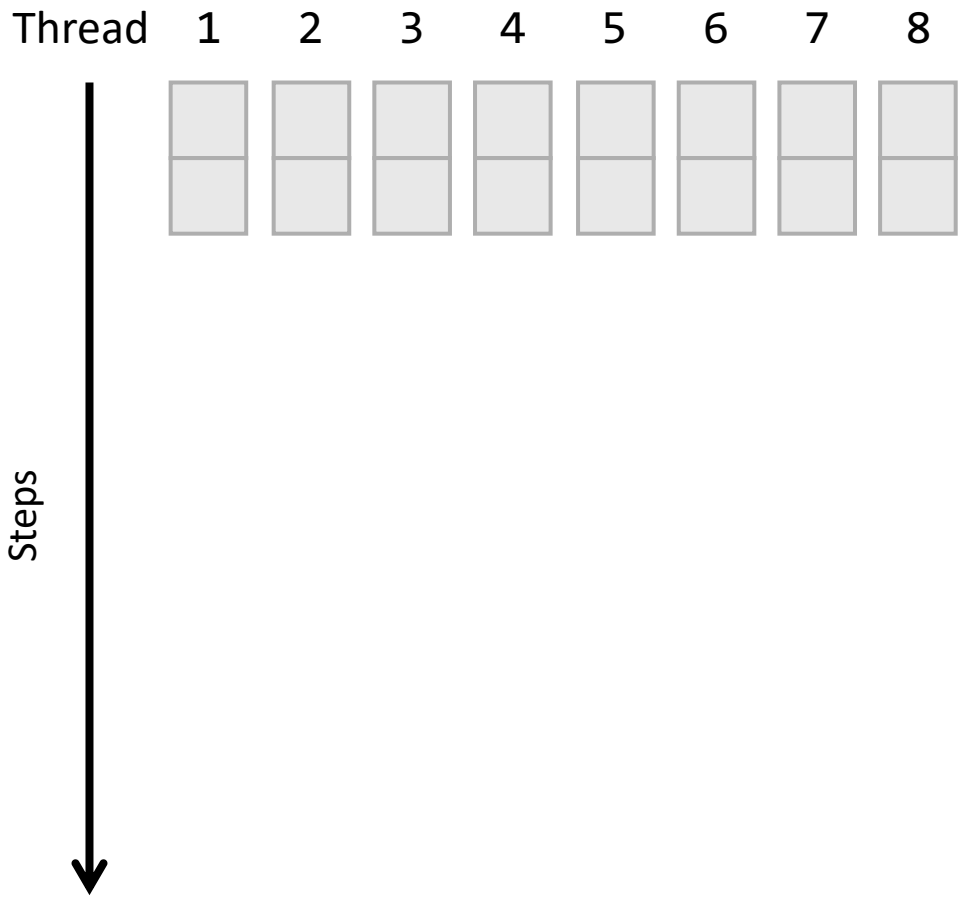
Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float x = A[i];
  if (x > 0) {
    x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



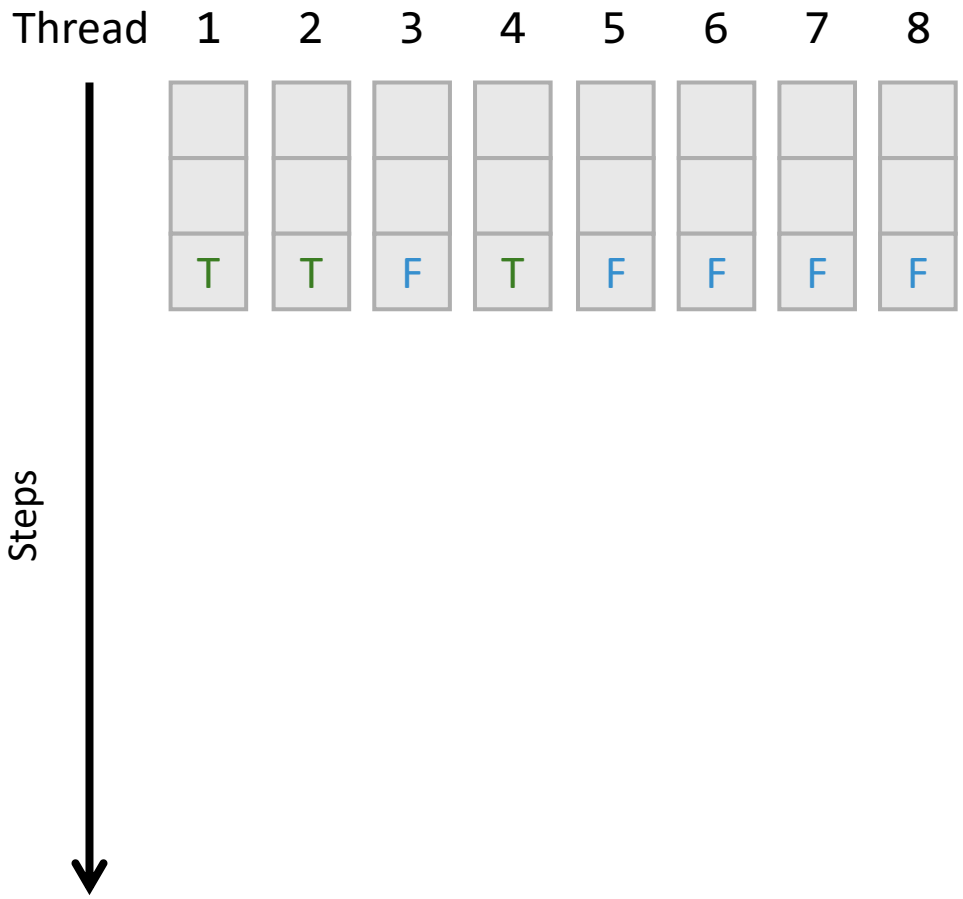
Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
⇒ float x = A[i];
  if (x > 0) {
    x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



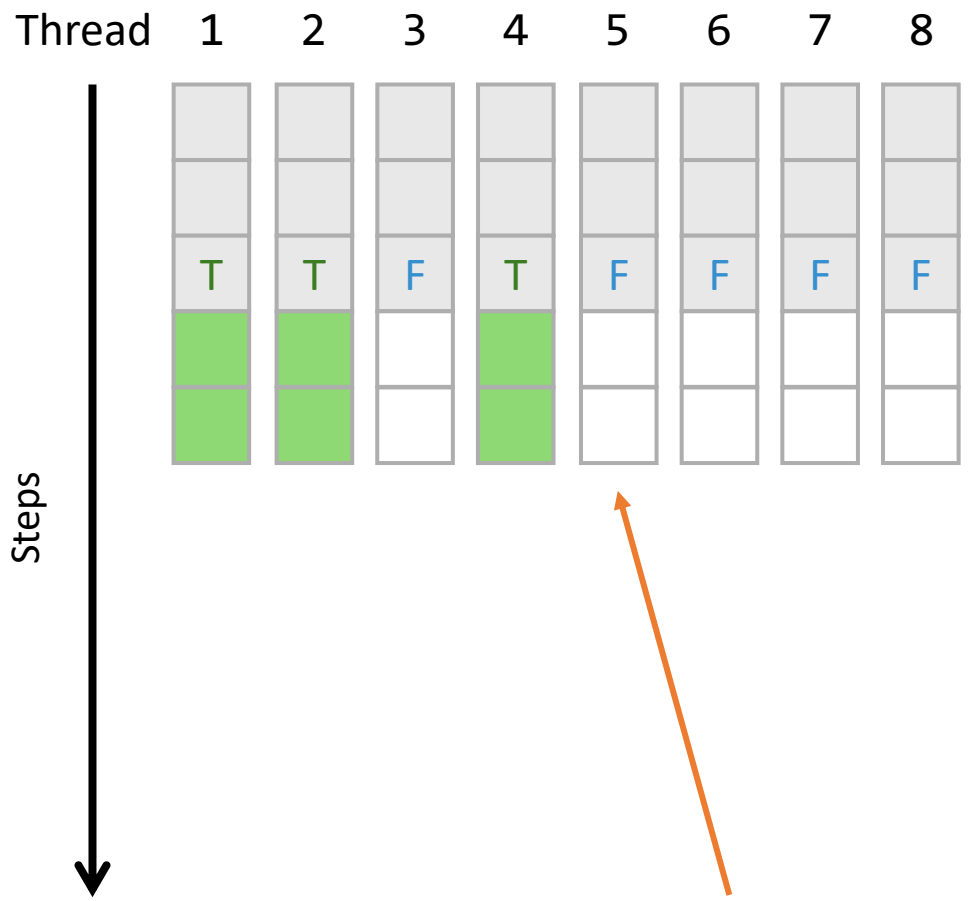
Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float x = A[i];
⇒ if (x > 0) {
    x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



Inactive lanes do not execute this branch.

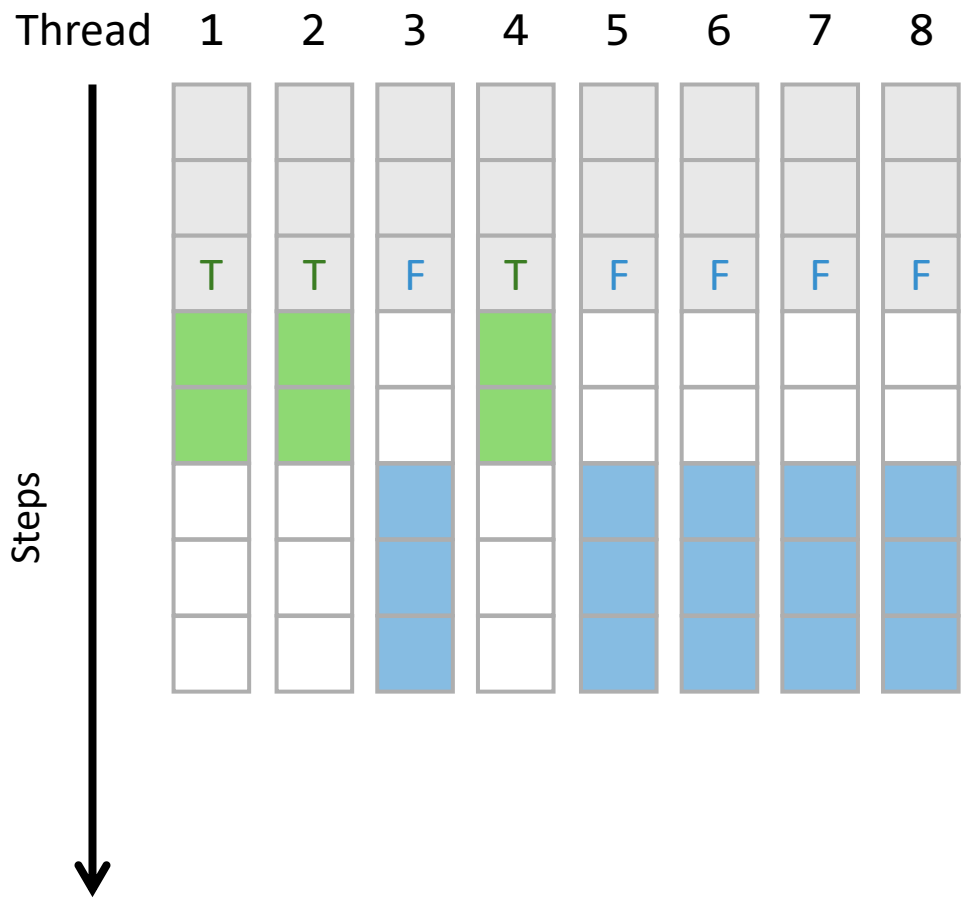
Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float x = A[i];
  if (x > 0) {
⇒   x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



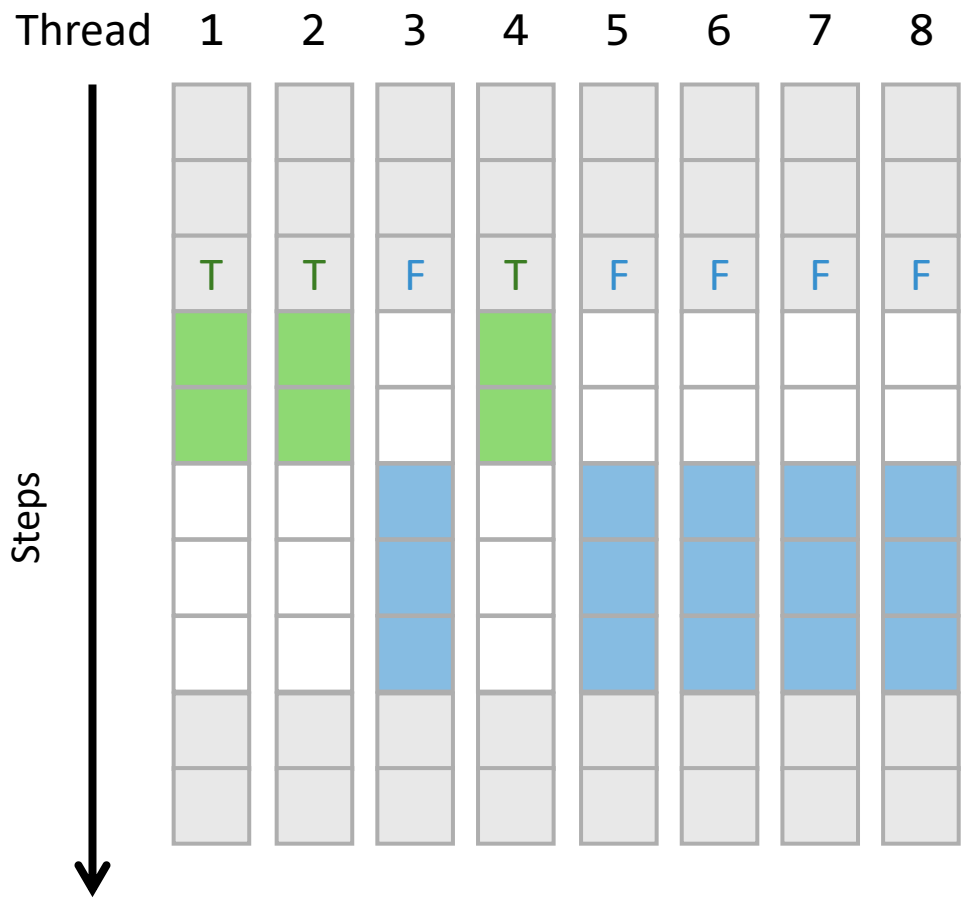
Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float x = A[i];
  if (x > 0) {
    x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

SIMT: Divergent Execution Overhead

One warp = 32 threads; 8 lanes shown. Schematic steps, not clock cycles. Exact launch size N.



Inputs for threads 1–8:
x = [1, 2, 0, 4, −1, −2, −3, −4]

```
__global__ void f(float A[N]) {
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  float x = A[i];
  if (x > 0) {
    x = 2.0f * x;
  } else {
    x = expf(x);
  }
  A[i] = x;
}
```

Kernel function

Terminology

Coherent execution within a warp

Active lanes follow the same instruction path.

This uses the execution lanes efficiently.

Divergent execution within a warp

Different paths execute with different active lanes.

Cost depends on the paths and the compiler.

Divergence Depends on the Mapping

Same 64 values: 32 positive and 32 nonpositive

Mixed assignment

Warp 0 — 16 if + 16 else

Warp 1 — 16 if + 16 else

Each warp mixes two paths.

Coherent assignment

Warp 0 — 32 if

Warp 1 — 32 else

Each warp follows one path.

Preserve original output indices. Rearrangement has a cost; this is not a guaranteed speedup.

Host Code: Allocation and Input

上海创智学院
Shanghai Innovation Institute

```
const int n = 1000;
const size_t bytes = size_t(n) * sizeof(float);
std::vector<float> a(n, 1.0f), b(n, 2.0f), c(n);
float *da, *db, *dc;
CHECK(cudaMalloc(&da, bytes));
CHECK(cudaMalloc(&db, bytes));
CHECK(cudaMalloc(&dc, bytes));
CHECK(cudaMemcpy(da, a.data(), bytes, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(db, b.data(), bytes, cudaMemcpyHostToDevice));
```

Host program, part 1 of 2. CHECK reports CUDA errors.

Host Code: Launch, Verify and Free

上海创智学院
Shanghai Innovation Institute

```
const int block = 256;
vec_add<<<(n + block - 1) / block, block>>>(da, db, dc, n);
CHECK(cudaGetLastError());
CHECK(cudaDeviceSynchronize());
CHECK(cudaMemcpy(c.data(), dc, bytes, cudaMemcpyDeviceToHost));
for (int i = 0; i < n; ++i) {
    if (c[i] != 3.0f) std::abort();
}
CHECK(cudaFree(da));
CHECK(cudaFree(db));
CHECK(cudaFree(dc));
```

Host program, part 2 of 2. Launch submission is asynchronous.

Error Checking and Input Coverage

上海创智学院
Shanghai Innovation Institute

```
inline void check(cudaError_t e) {
    if (e != cudaSuccess) {
        std::fprintf(stderr, "%s\n",
                     cudaGetErrorString(e));
        std::exit(EXIT_FAILURE);
    }
}
#define CHECK(call) check(call)
```

1. Check API return values
2. Check launch and execution
3. Compare against a reference

Test tiny inputs, exact tiles and partial tiles before timing.

Checkpoint: CUDA Work Assignment

上海创智学院
Shanghai Innovation Institute

n = 1000, blockDim.x = 256

1. How many blocks and threads are launched?
2. What does block 3, thread 10 compute?
3. When may the CPU safely use the result?

Explain the boundary guard and the completion dependency.

03

Layouts and Memory Access

Logical Shape and Physical Storage

上海创智学院
Shanghai Innovation Institute

Logical matrix

0	1	2	3
4	5	6	7
8	9	10	11

shape = (3, 4)
strides = (4, 1)

Same linear storage

0	1	2	3	4	5	6	7	8	9	10	11

Element offset = row * 4 + col. Byte offset also includes element size.

A Transposed View

上海创智学院
Shanghai Innovation Institute

Transposed view

0	4	8
1	5	9
2	6	10
3	7	11

shape = (4, 3)
strides = (1, 4)

Same linear storage

0	1	2	3	4	5	6	7	8	9	10	11

A view changes indexing metadata. A materialized transpose writes new storage.

Thread Mapping Meets Data Layout

上海创智学院
Shanghai Innovation Institute

Lanes move across columns

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Offsets 0, 1, 2, 3

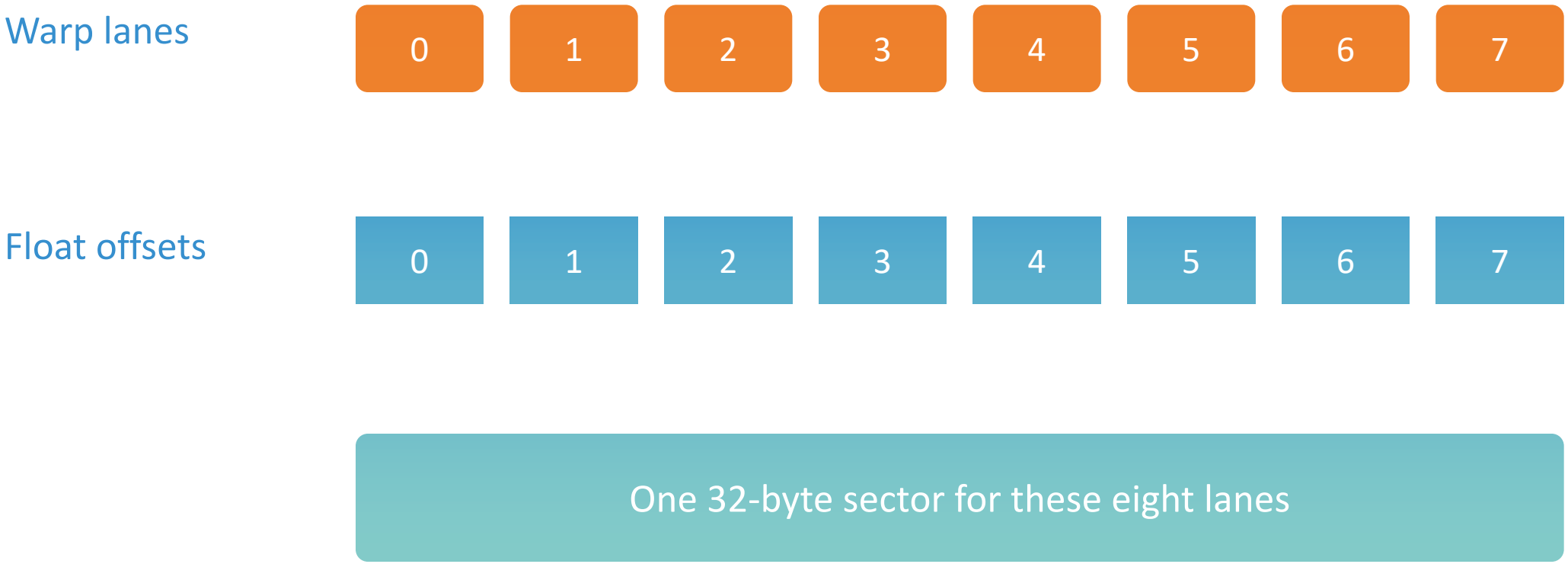
Lanes move across rows

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Offsets 0, 4, 8, 12

Memory layout and thread assignment jointly determine the access pattern.

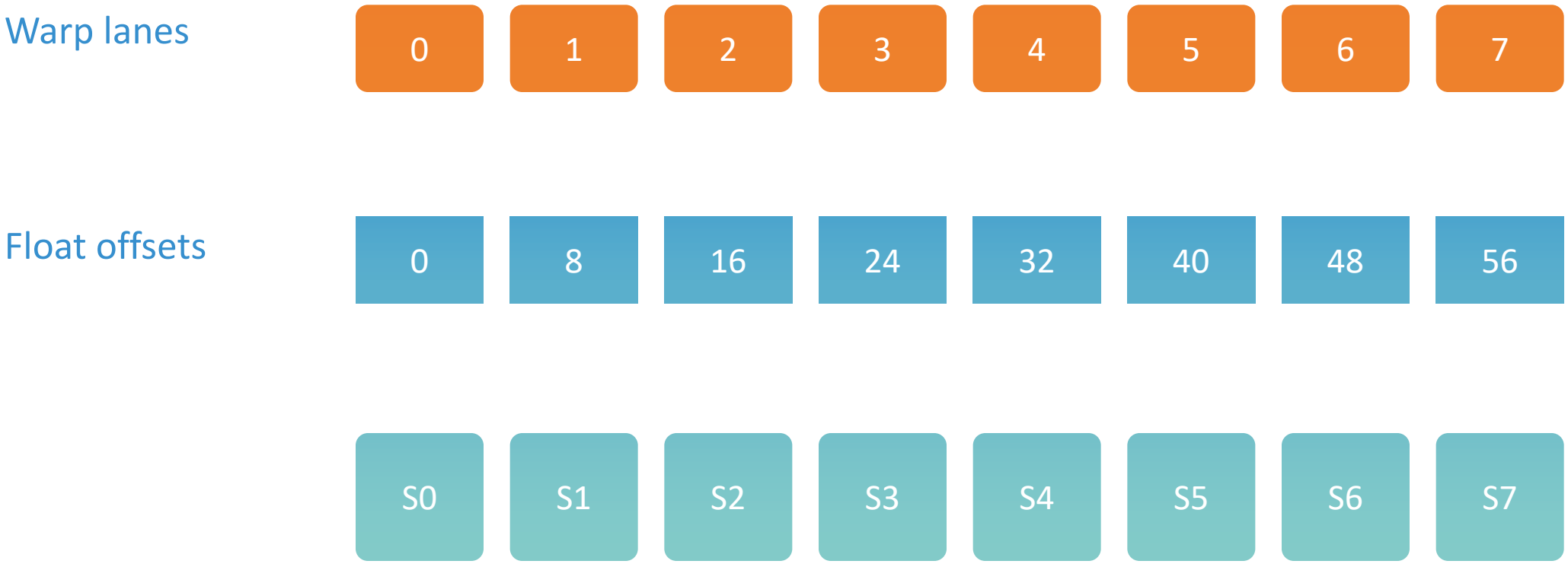
Coalesced Global-Memory Access



First 8 of 32 lanes shown. The base address is 32-byte aligned.

32 lanes × 4 bytes = 128 useful bytes in four aligned 32-byte sectors.

Strided Global-Memory Access



First 8 of 32 lanes shown. The base address is 32-byte aligned.

Stride = 8 floats: the same 128 useful bytes touch 32 sectors.

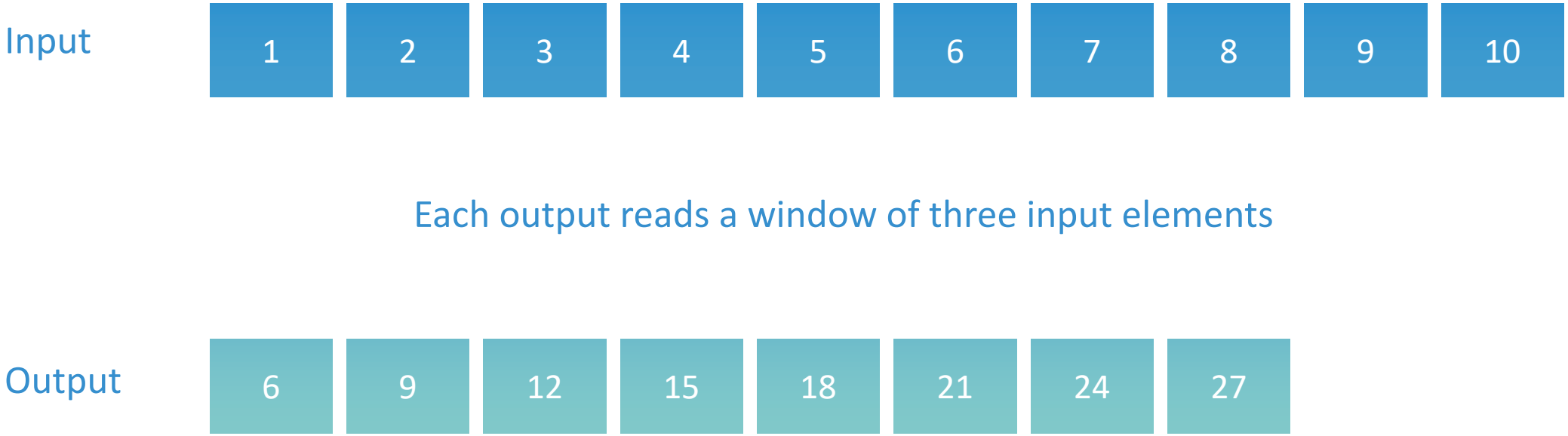
Requested Bytes and Sector Bytes

上海创智学院
Shanghai Innovation Institute

One warp load	Contiguous	Stride = 8 floats
Useful data	128 B	128 B
32-byte sectors	4	32
Sector bytes	128 B	1024 B
Useful / sector bytes	100%	12.5%

CC ≥ 6.0 model. Cache behavior and achieved throughput require measurement.

1D Convolution: Three-Point Filter



Input length = n + 2. Output length = n. Filter = [1, 1, 1].

1D Convolution: Naive Kernel

n = 4 outputs; input x = [1,2,3,4,5,6] has n+2 values

```
int i=blockIdx.x*blockDim.x+threadIdx.x;
if (i<n)
  y[i]=x[i]+x[i+1]+x[i+2];
```

Thread 1 reads



y[1] = 2 + 3 + 4 = 9

Thread 2 reads

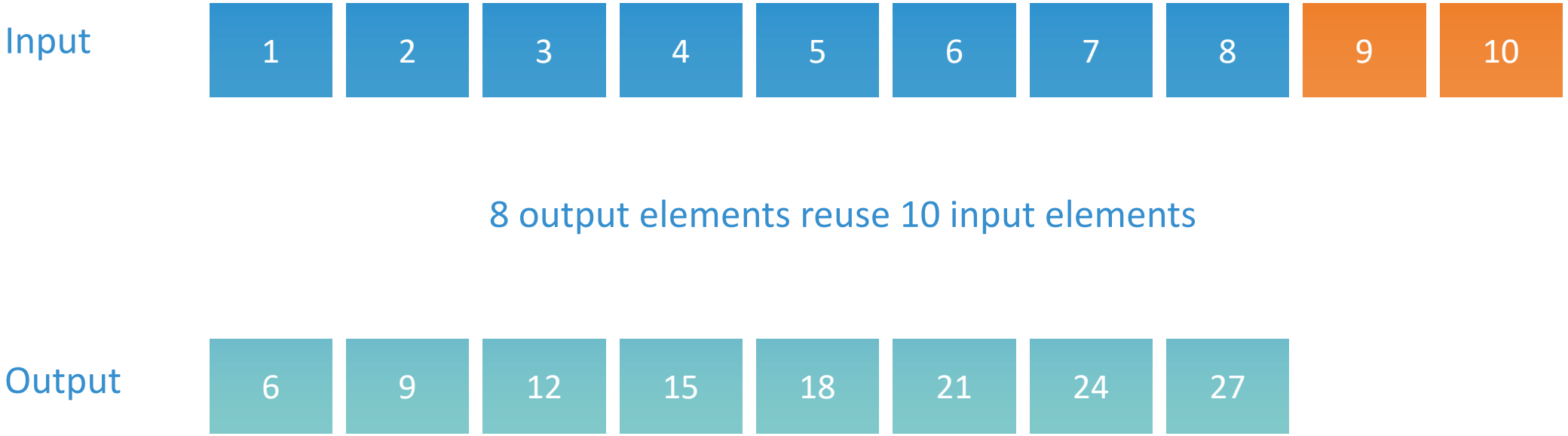


y[2] = 3 + 4 + 5 = 12

Neighboring outputs request
some of the same input values.

Per output: 3 global input reads (12 B), 1 output write (4 B). Cache effects are excluded.

1D Convolution: The Input Tile



B outputs need B + 2 unique input elements. Here B = 8.

1D Convolution: Cooperative Loading

n = B = 4: produce y = [6,9,12,15] from six input values

```
__shared__ float tile[B+2];
int t=threadIdx.x, base=blockIdx.x*B;
for(int j=t; j<B+2; j+=B)
  tile[j]=(base+j<n+2)?x[base+j]:0;
__syncthreads();
if(base+t<n)
  y[base+t]=tile[t]+tile[t+1]+tile[t+2];
```

Cooperative input loads

T0: x[0]=1, x[4]=5

T1: x[1]=2, x[5]=6

T2: x[2]=3

T3: x[3]=4



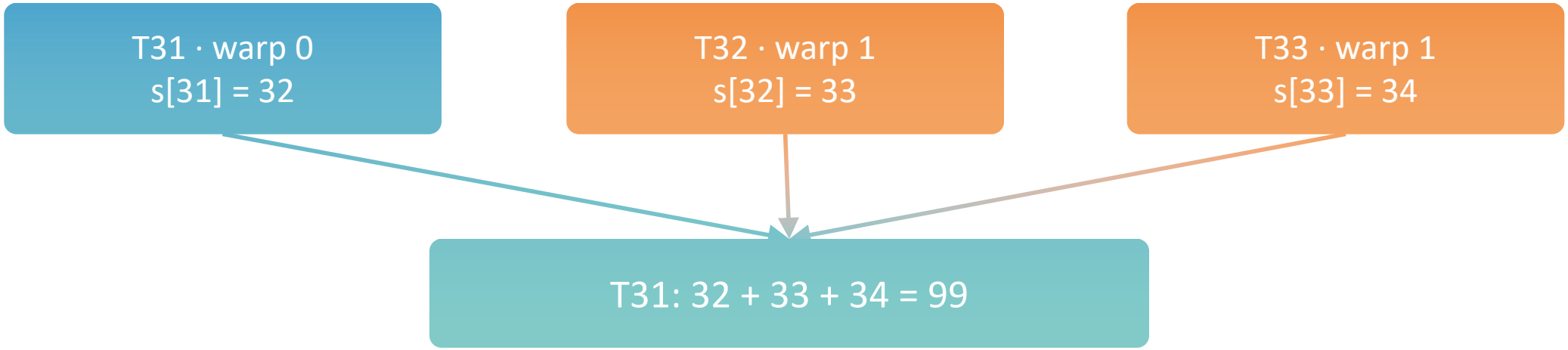
Block input: 12 → 6 global floats (48 → 24 B). Extra shared work: 6 writes, 12 reads, 1 barrier.

1D Convolution: Who Produces My Inputs?

上海创智学院
Shanghai Innovation Institute

64-thread block; base = 0; x[j] = j + 1

```
y[t] = s[t] + s[t + 1] + s[t + 2];
```



T63 also needs halo values 65 and 66 loaded by T0 and T1.

Shared storage permits access across threads; it does not by itself order their writes and reads.

1D Convolution: Why __syncthreads()?

上海创智学院
Shanghai Innovation Institute

```
s[t] = x[base + t];
// load halo as needed
__syncthreads();
if (base + t < n)
  y[base + t] = s[t] + s[t+1] + s[t+2];
```

Without the barrier

T31 may read s[32] / s[33] before warp 1 writes them.

With the barrier



One block only. An early read has no guaranteed value; it is not necessarily zero.

1D Convolution: The Last Partial Block

上海创智学院 Shanghai Innovation Institute

B = 4, n = 6, input length = 8; last block has base = 4

```
for (int j = t; j < B+2; j += B)
  s[j] = (base+j < n+2)
       ? x[base+j] : 0;
__syncthreads();
if (base+t < n)
  y[base+t] = s[t]+s[t+1]+s[t+2];
```

Shared values



T2 / T3 have no output,
but must load 7 / 8.

T0 writes y[4] = 18
T1 writes y[5] = 21

All four threads load and synchronize; only two write outputs.

Do not return early based only on the output index: an inactive output thread can still be a producer.

1D Convolution: Shared-Memory Kernel

上海创智学院
Shanghai Innovation Institute

```
template<int B>
__global__ void conv1d(const float* x, float* y, int n) {
  __shared__ float s[B + 2];
  int t = threadIdx.x, base = blockIdx.x * B;
  for (int j = t; j < B + 2; j += B)
    s[j] = (base+j < n+2) ? x[base+j] : 0.0f;
  __syncthreads();
  if (base+t < n)
    y[base+t] = s[t] + s[t+1] + s[t+2];
}
// n > 0; input has n+2 values; output has n values
conv1d<128><<<(n+127)/128, 128>>>(x, y, n);
```

Full block: 3B → B + 2 global input values; B outputs unchanged.

Same unit-weight sum as the naive kernel. Added costs: shared writes, shared reads, and a block barrier.

Reuse Has Benefits and Costs

上海创智学院
Shanghai Innovation Institute

For B = 128 full outputs	Naive	Shared-memory version
Scalar global input loads	384	130
Output stores	128	128
Shared-memory operations	None	Loads and stores
Block barrier	None	One

Fewer logical loads do not by themselves prove a faster kernel.

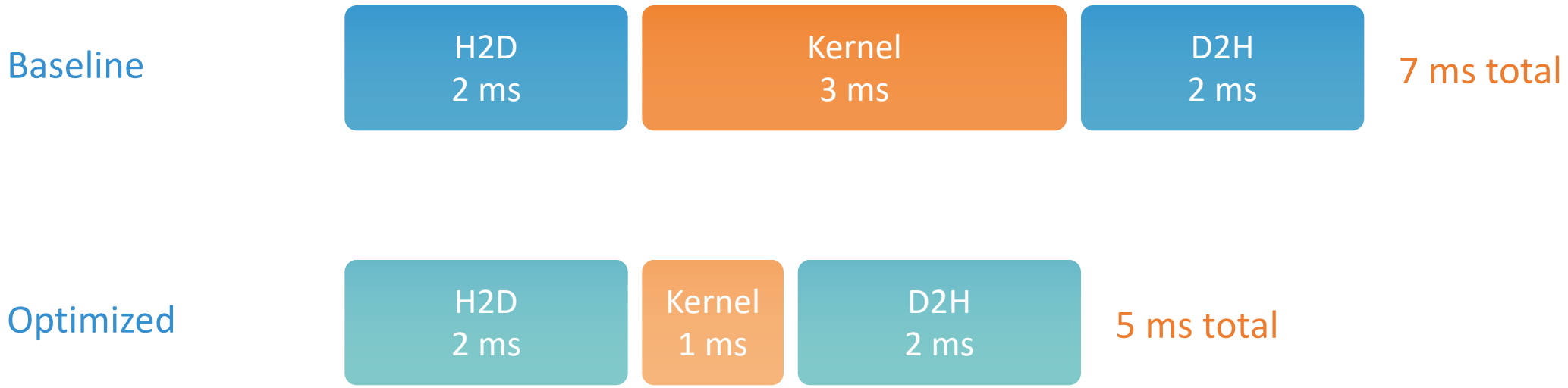
04

Measuring Performance

Kernel Time and End-to-End Time

上海创智学院
Shanghai Innovation Institute

Same workload, no overlap; illustrative times



Kernel: 3 / 1 = 3×

End-to-end: 7 / 5 = 1.4×

Scope: H2D + kernel + D2H. Allocation and other host overhead are excluded.

Timing with CUDA Events

上海创智学院
Shanghai Innovation Institute

```
cudaEvent_t start, stop;
CHECK(cudaEventCreate(&start));
CHECK(cudaEventCreate(&stop));
CHECK(cudaEventRecord(start));
for (int r = 0; r < repeats; ++r)
    vec_add<<<grid, block>>>(da, db, dc, n);
CHECK(cudaGetLastError());
CHECK(cudaEventRecord(stop));
CHECK(cudaEventSynchronize(stop));
float ms;
CHECK(cudaEventElapsedTime(&ms, start, stop));
CHECK(cudaEventDestroy(start));
CHECK(cudaEventDestroy(stop));
```

Warm up first. Report ms / repeats and the repetition protocol.

A Reproducible Comparison

上海创智学院
Shanghai Innovation Institute

Same GPU • 1024 × 1024 • FP32 • kernel-only timing



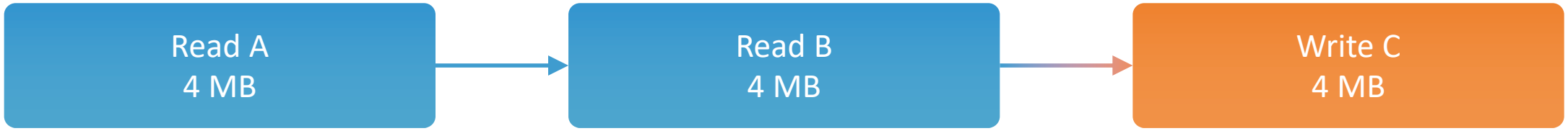
Baseline (ms)	3.0	3.1	2.9	median 3.0
Optimized (ms)	2.4	2.5	2.3	median 2.4

Comparable speedup: 3.0 / 2.4 = 1.25×

Illustrative data. Use the same inputs, repetition protocol and timing scope in every round.

A Performance Bound

One million FP32 additions: c[i] = a[i] + b[i]



1 MFLOP / 12 MB = 1/12 FLOP per byte

Assumed FP32 peak: 20 TFLOP/s
Assumed memory bandwidth: 120 GB/s

120 GB/s × 1/12 FLOP/B

$$P \leq 10\ GFLOP/s$$

Assume 12 MB moves at device memory. Decimal units; an upper bound, not a timing result.

Materializing a Matrix Transpose

Input A

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

(row, col)
becomes
(col, row)

Output B

0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

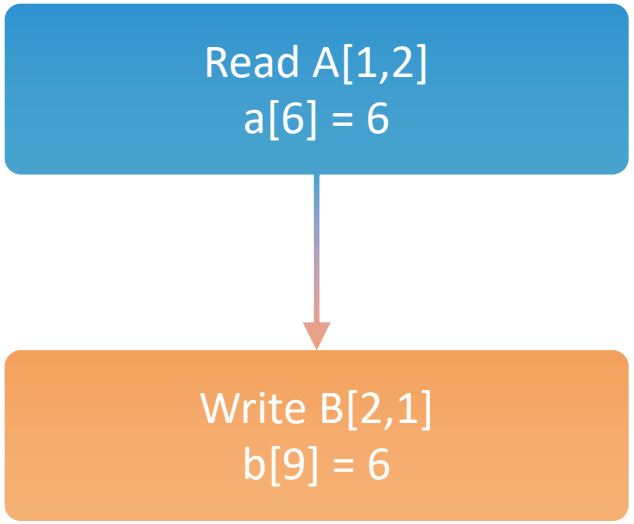
B[col, row] = A[row, col], with separate row-major input and output.

Transpose: A Correct Baseline

上海创智学院
Shanghai Innovation Institute

4 × 4 row-major matrix: follow input A[1,2] = 6

```
int col=blockIdx.x*blockDim.x+threadIdx.x;
int row=blockIdx.y*blockDim.y+threadIdx.y;
if (row<rows && col<cols)
  b[col*rows+row]=a[row*cols+col];
```



For row = 1, col = 0..3:
read offsets 4,5,6,7
write offsets 1,5,9,13

Core CUDA code. The element count is unchanged; the write addresses are strided.

A Tile Reorders the Data

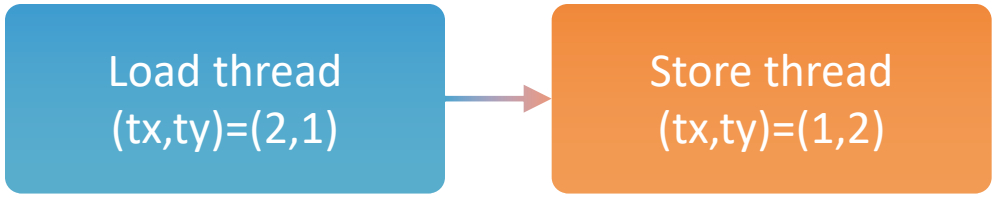
上海创智学院
Shanghai Innovation Institute

Illustrative 4 × 4 tile: follow the value 6

A and shared tile

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

tile[1][2] = 6



Barrier before the exchange

Output B

0	4	8	12
1	5	9	13
2	6	10	14
3	7	11	15

```
tile[ty][tx] = A[ty][tx];
sync; B[ty][tx] = tile[tx][ty];  // one 4 x 4 tile
```

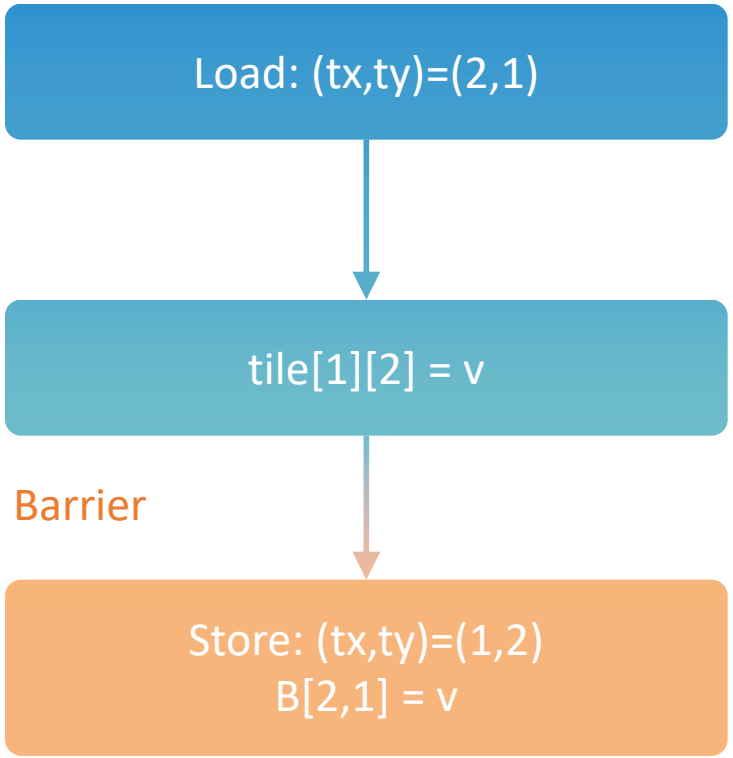
Per tile: 16 global reads + 16 global writes; also 16 shared writes + 16 shared reads.

Transpose Through Shared Memory

上海创智学院 Shanghai Innovation Institute

block = (32,8), blockIdx = (0,0): follow tile[1][2]

```
__shared__ float tile[32][32];
int tx=threadIdx.x, ty=threadIdx.y;
int x=blockIdx.x*32+tx;
int y=blockIdx.y*32+ty;
for(int j=0;j<32;j+=8)
  if(x<cols && y+j<rows)
    tile[ty+j][tx]=a[(y+j)*cols+x];
__syncthreads();
x=blockIdx.y*32+tx; y=blockIdx.x*32+ty;
for(int j=0;j<32;j+=8)
  if(x<rows && y+j<cols)
    b[(y+j)*rows+x]=tile[tx][ty+j];
```

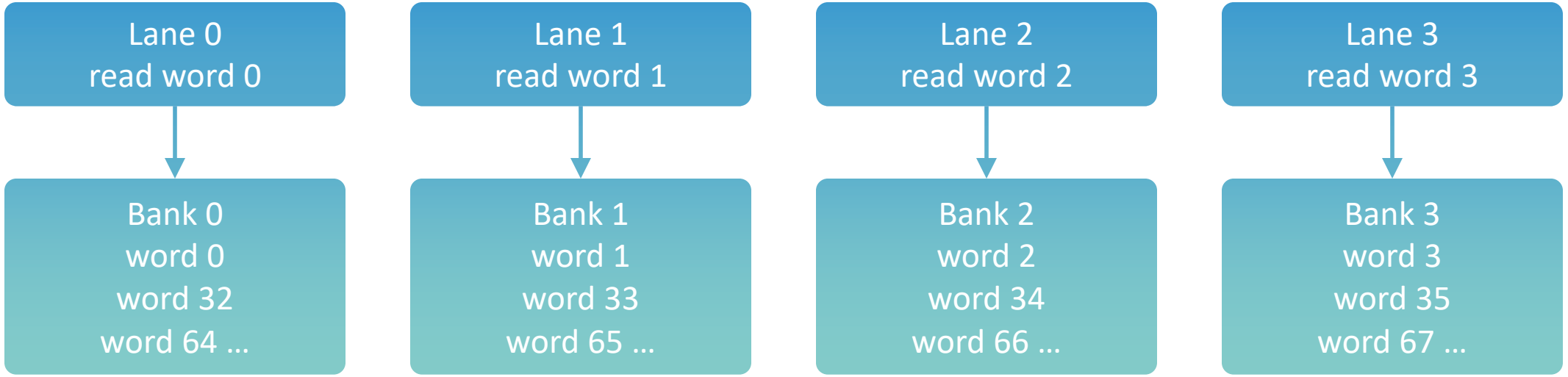


Full tile: 4096 B global read + 4096 B global write; also 1024 shared stores + 1024 shared loads.

Shared Memory Has Banks

32 independent banks provide parallel bandwidth.

Each bank has limited bandwidth: 32 bits/cycle in this model.



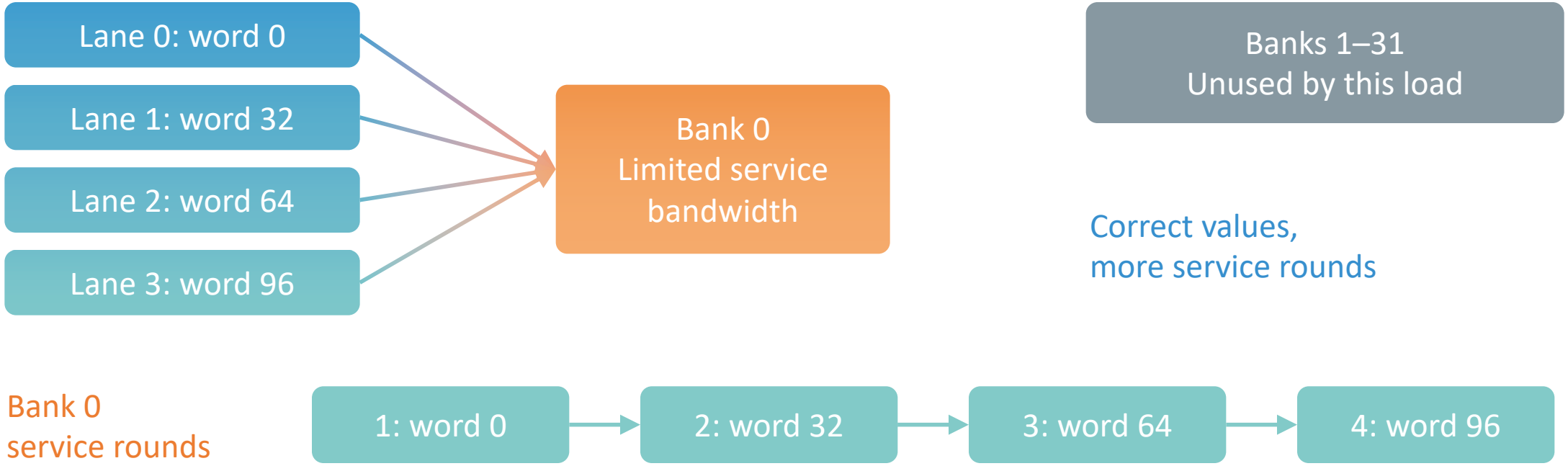
$$\text{bank} = \text{wordAddress} \bmod 32$$

CC 5.x+ 32-bank model; aligned scalar 32-bit reads. Four lanes/banks shown; bank contents are word addresses.

Column Access Can Conflict

```
__shared__ float tile[32][32];  float v = tile[lane][0];
```

Different words contend for bank 0's limited service bandwidth.



Four lanes shown; all 32 lanes give a 32-way conflict. Service rounds are not the full load latency in cycles.

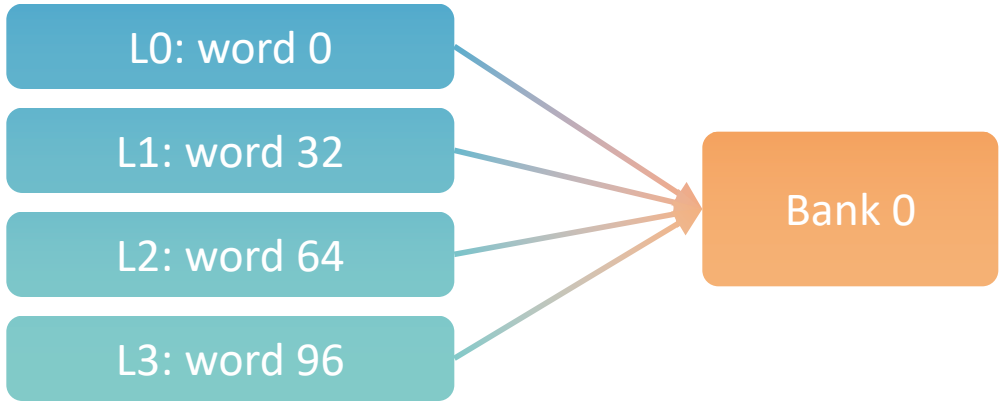
Padding Changes the Bank Mapping

Same column read: lane l reads tile[l][0] (lanes 0–3 shown)

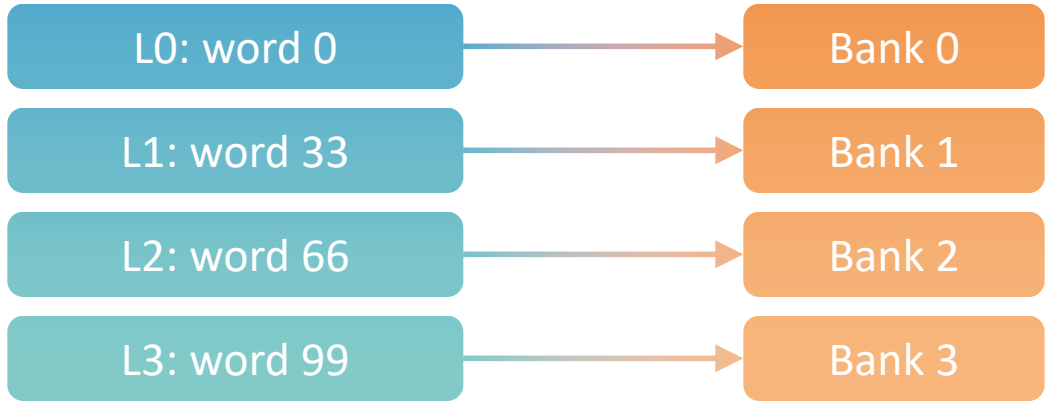
```
float tile[32][32];
```

```
float tile[32][33];
```

Different words, one bank



Different words, distinct banks



4096 B shared → 4224 B shared (+128 B)

32-bank, scalar 32-bit model. The column now uses 32 independent banks in one service round.

Bank Conflicts, Broadcast and Races

上海创智学院
Shanghai Innovation Institute

Different words	Lane 0 read word 0	Lane 1 read word 32	Same bank: conflict
Same word	Lane 0 read word 0	Lane 1 read word 0	Read broadcast
Conflicting writes	Lane 0 word 0 = 10	Lane 1 word 0 = 20	Data race

32-bank, 32-bit model. A barrier changes ordering; it does not change the bank mapping.

Layout Checkpoint

One warp reads float tile[32][LD]. Lane l reads tile[l][0].

1. For LD = 32, which banks are used?
2. For LD = 33, which banks are used?
3. Does padding guarantee a faster complete kernel?

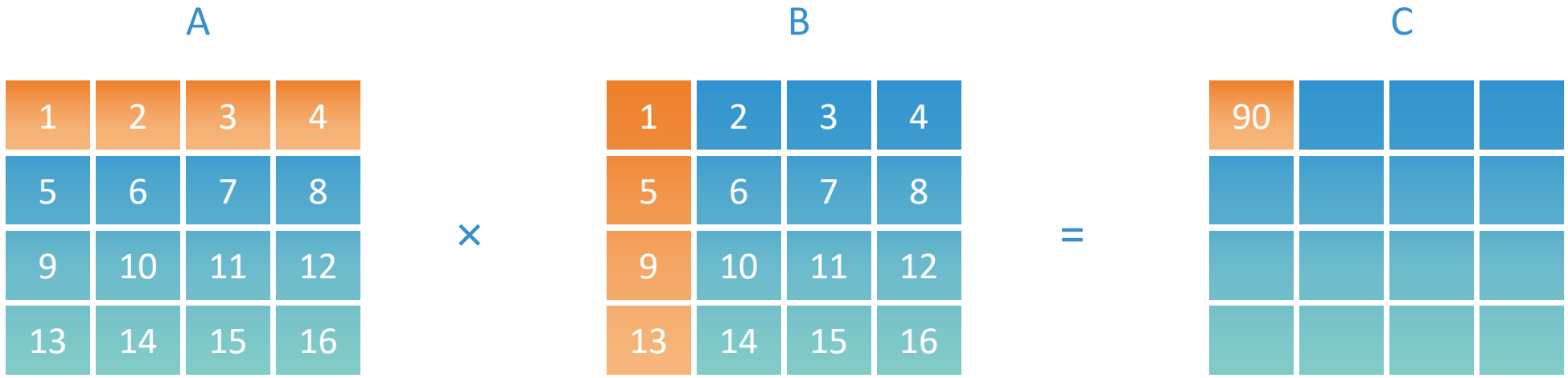
Use bank = (l * LD) mod 32 under the stated 32-bit access model.

05

Matrix Multiplication

Matrix Shapes and Indexing

One continuing example: M = N = K = 4



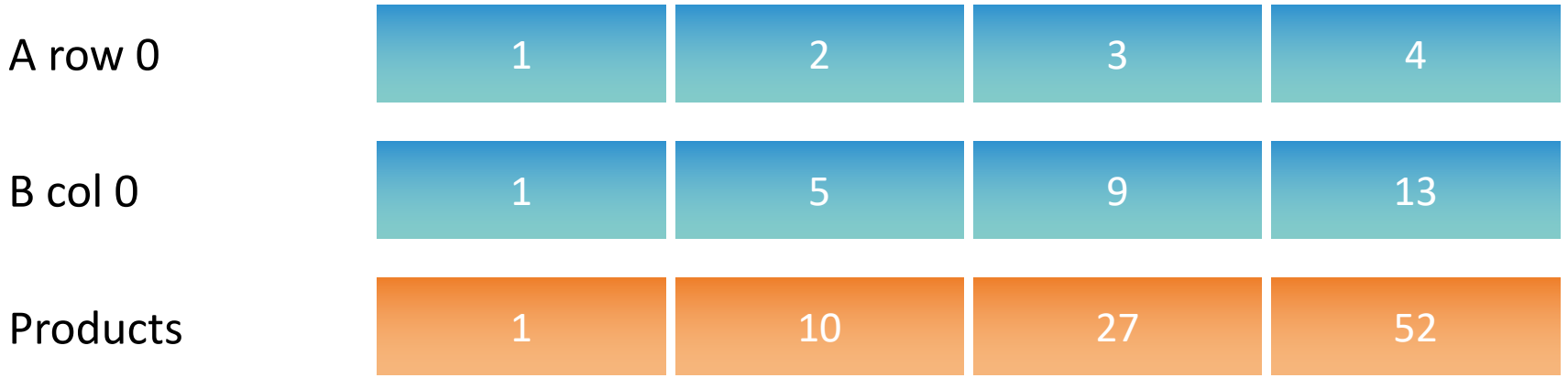
$$C_{00} = 1\times1 + 2\times5 + 3\times9 + 4\times13 = 90$$

Same row-major matrices on the following pages. A small T = 2 tile makes every move visible.

One Thread Owns One Output

上海创智学院
Shanghai Innovation Institute

Thread (row, col) = (0, 0) computes C[0,0]




```
sum = 0;
for k = 0..3: sum += A[0,k] * B[k,0];
C[0,0] = sum;  // 90
```

Global input
8 floats = 32 B
Output: 4 B

sum stays private: 0 → 1 → 11 → 38 → 90. Counts are explicit requests before caches.

Naive Matrix Multiplication

上海创智学院
Shanghai Innovation Institute

One output thread: row = 0, col = 1, M = N = K = 4

```
__global__ void matmul_naive(
  const float *a, const float *b,
  float *c, int M, int N, int K) {
  int row=blockIdx.y*blockDim.y+threadIdx.y;
  int col=blockIdx.x*blockDim.x+threadIdx.x;
  if(row>=M || col>=N) return;
  float sum=0;
  for(int k=0;k<K;++k)
    sum+=a[row*K+k]*b[k*N+col];
  c[row*N+col]=sum;
}
```

A row: [1,2,3,4]
B col: [2,6,10,14]

k=0: 0 + 1×2 = 2

k=1: 2 + 2×6 = 14

k=2: 14 + 3×10 = 44

k=3: 44 + 4×14 = 100

Read 8 floats = 32 B
Write C[0,1] once = 4 B

Whole 4 × 4 matrix: 128 scalar input requests (512 B), 16 output stores (64 B), before caches.

Repeated Input Requests

上海创智学院
Shanghai Innovation Institute

Four output threads, same left-top 2 × 2 output tile

A rows needed

1	2	3	4
5	6	7	8

B columns needed

1	2
5	6
9	10
13	14

Unique inputs
8 A + 8 B
16 floats = 64 B

```
for each of 4 outputs:
  for k = 0..3: load A[row,k] and B[k,col];
```

Actual source requests: 4 outputs × 4 k steps × 2 loads = 32 floats = 128 B, before caches.

A Block Owns an Output Tile

上海创智学院
Shanghai Innovation Institute

One block: four output threads, T = 2, K = 4

A: two rows

1	2	3	4
5	6	7	8

B: two columns

1	2
5	6
9	10
13	14

Output ownership

T00	T01
T10	T11

Inputs: 64 B instead of 128 B. Output: 16 B in both versions.

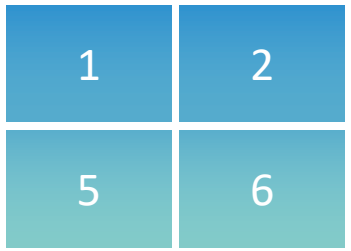
```
for k0 = 0, 2:
  load A[0:2,k0:k0+2] -> As; load B[k0:k0+2,0:2] -> Bs;
  sync; update four private sums; sync;
store four C values;
```

Global requests per output tile. Exact tiling; source-level counts, excluding cache effects.

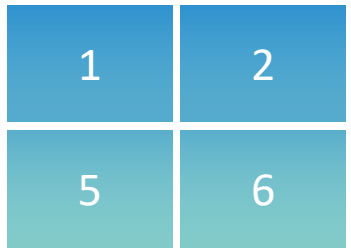
One Tile of the K Dimension

First K stage: k0 = 0 selects k = 0 and 1

As = A[0:2,0:2]



Bs = B[0:2,0:2]



Per stage
8 global reads = 32 B
8 shared writes
Shared capacity: 32 B

```
As[ty][tx] = A[row][k0 + tx];
Bs[ty][tx] = B[k0 + ty][col];
sync; for kk = 0,1: sum += As[ty][kk] * Bs[kk][tx];
sync;  // then reuse the same buffers with k0 = 2
```

Four T = 2 threads keep their private sums while the K stage advances.

Threads Cooperatively Load the Tiles

上海创智学院
Shanghai Innovation Institute

k0 = 0: each thread fills one element of each shared tile




```
As[ty][tx] = A[row][k0 + tx];
Bs[ty][tx] = B[k0 + ty][col];
```

Each thread: 2 global reads + 2 shared writes. Block total: 8 reads + 8 writes per stage.

Read the Shared Tiles and Accumulate

Four private accumulators survive both K stages

Before K loop

0	0
0	0

After k0 = 0

11	14
35	46

After k0 = 2

90	100
202	228

T00: 1×1 + 2×5 = 11; then add 3×9 + 4×13 = 79

```
for kk = 0,1: sum += As[ty][kk] * Bs[kk][tx];
// After all K stages: write each sum to C once.
```

Per stage: 4 shared reads/thread = 16/block. Final global output: 4 floats = 16 B.

A Second Barrier Protects Buffer Reuse

A shared slot changes from 1 (tile k) to 9 (tile k+1)

Without the second barrier



The old tile has already been overwritten.

```
read tile k and update sum;
__syncthreads();  // all current reads finish
load tile k+1 into the same shared storage;
```

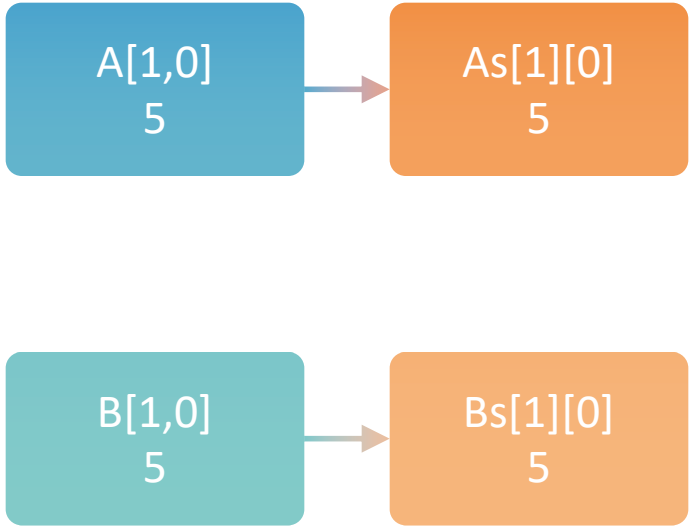
A multi-warp block. Barrier 1: produce → consume. Barrier 2: consume → overwrite.

Tiled Matrix Multiplication: Load

Hand trace: T = 2, k0 = 0, (ty,tx) = (1,0)

```
__shared__ float As[T][T], Bs[T][T];
int tx=threadIdx.x, ty=threadIdx.y;
int row=blockIdx.y*T+ty;
int col=blockIdx.x*T+tx;
float sum=0;
for(int k0=0;k0<K;k0+=T) {
  As[ty][tx]=(row<M && k0+tx<K)
      ? a[row*K+k0+tx] : 0.0f;
  Bs[ty][tx]=(k0+ty<K && col<N)
      ? b[(k0+ty)*N+col] : 0.0f;
  // Compute phase on the next page.
```

row = 1, col = 0



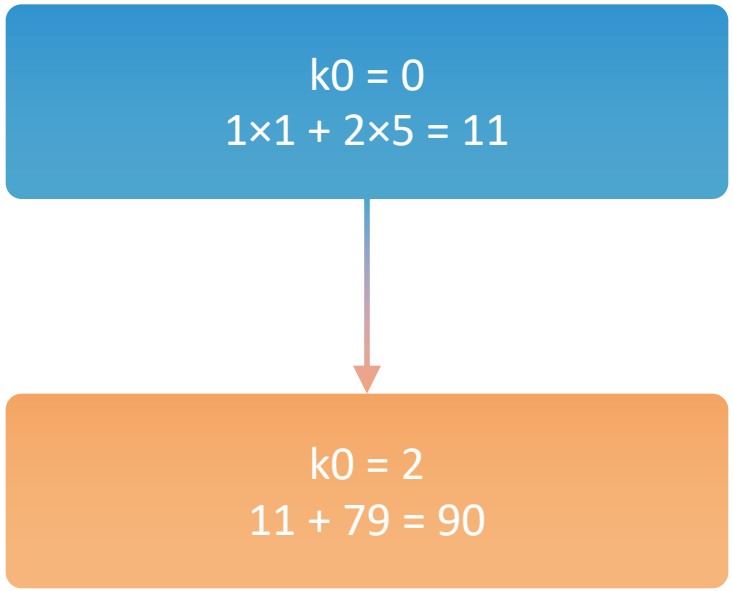
Per thread/stage: 2 global reads + 2 shared stores. T = 2 illustrates the same code used with T = 16.

Tiled Matrix Multiplication: Compute

上海创智学院
Shanghai Innovation Institute

Compute trace: (ty,tx) = (0,0), sum survives the K loop

```
  __syncthreads(); // loads -> reads
  for(int kk=0;kk<T;++kk)
    sum+=As[ty][kk]*Bs[kk][tx];
  __syncthreads(); // reads -> overwrite
}
if(row<M && col<N)
  c[row*N+col]=sum;
```



Keep the private sum.
Replace only the shared input tiles.

Final write: C[0,0] = 90

T = 2: each stage reads 4 shared floats per thread. Only one global output write per thread.

A Logical Traffic Model

上海创智学院
Shanghai Innovation Institute

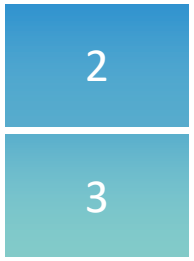
Square matrices: N = 128	Naive	Tiled, T = 16
Input scalar requests	$2 \times N^3$	$2 \times N^3 / T$
Input bytes, FP32	16 MiB	1 MiB
Output bytes, FP32	64 KiB	64 KiB

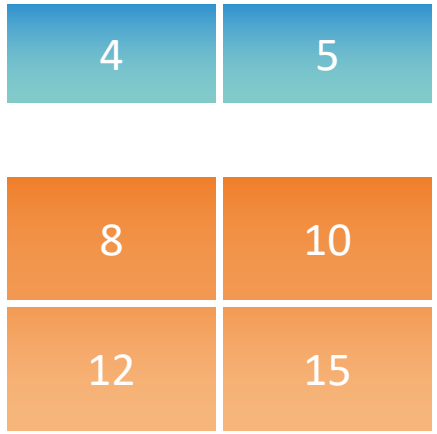
Algorithmic requests, exact tiling, cache effects excluded.

A Thread Can Own Several Outputs

One thread, one k step: a = [2,3], b = [4,5]

A values





4 input values
4 multiply-adds
4 private accumulators

```
acc00 += a0*b0;  acc01 += a0*b1;
acc10 += a1*b0;  acc11 += a1*b1;
```

Global reads in this example: 4 floats (16 B), versus 8 floats (32 B) for four separate threads.

A 2 × 2 Register Tile

上海创智学院
Shanghai Innovation Institute

Interior tile: one thread owns four distinct output elements

```
float acc[2][2] = {};
for(int k=0;k<K;++k) {
  float a0=a[r0*K+k], a1=a[(r0+1)*K+k];
  float b0=b[k*N+c0], b1=b[k*N+c0+1];
  acc[0][0]+=a0*b0; acc[0][1]+=a0*b1;
  acc[1][0]+=a1*b0; acc[1][1]+=a1*b1;
}
```

One k step, acc starts at 0
a0 = 2, a1 = 3
b0 = 4, b1 = 5

8	10
12	15

Input per k: 4 floats (16 B), versus 8 floats (32 B) for four separate output threads. Edge guards omitted.

Block Reuse and Thread Reuse

上海创智学院
Shanghai Innovation Institute

Same 4 × 4 block tile; change only output ownership

16 threads: 1 output each

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

4 threads: 2 × 2 outputs each

0	0	1	1
0	0	1	1
2	2	3	3
2	2	3	3

Per kk: 32 shared reads → 16 shared reads

```
load A and B block tiles -> shared; sync;
for kk: load av[0:2] and bv[0:2] from shared;
        for i,j = 0,1: acc[i][j] += av[i] * bv[j];
```

For K = 4: global input stays 32 floats (128 B); global output stays 16 floats (64 B).

Larger Tiles Have Resource Costs

上海创智学院
Shanghai Innovation Institute

256 threads per block	16 × 16 output tile	32 × 32 output tile
Outputs per thread	1	4 (2 × 2)
FP32 shared A + B	2 KiB	8 KiB
Private accumulators	1 per thread	4 per thread
Global reads per output	32 floats = 128 B	16 floats = 64 B

K = 256; square input tiles T = 16 or 32. Global input per output = 2K/T values, before caches.

Resource-Limit Exercise

上海创智学院
Shanghai Innovation Institute

Hypothetical SM: 64 KiB shared memory, at most 64 resident warps.

1. Each block has 256 threads and uses 16 KiB shared memory.
2. How many blocks and warps can shared memory support?
3. What changes if shared memory rises to 32 KiB per block?

Ignore allocation granularity. Assume other limits are not tighter.

Matrix Multiplication: What Changed?

上海创智学院
Shanghai Innovation Institute

Version	Reuse mechanism	Main new cost
One output per thread	Private accumulator	Repeated input requests
Shared-memory tiling	Reuse across block threads	Barriers and shared capacity
Register tiling	Reuse across thread outputs	Register capacity / pressure

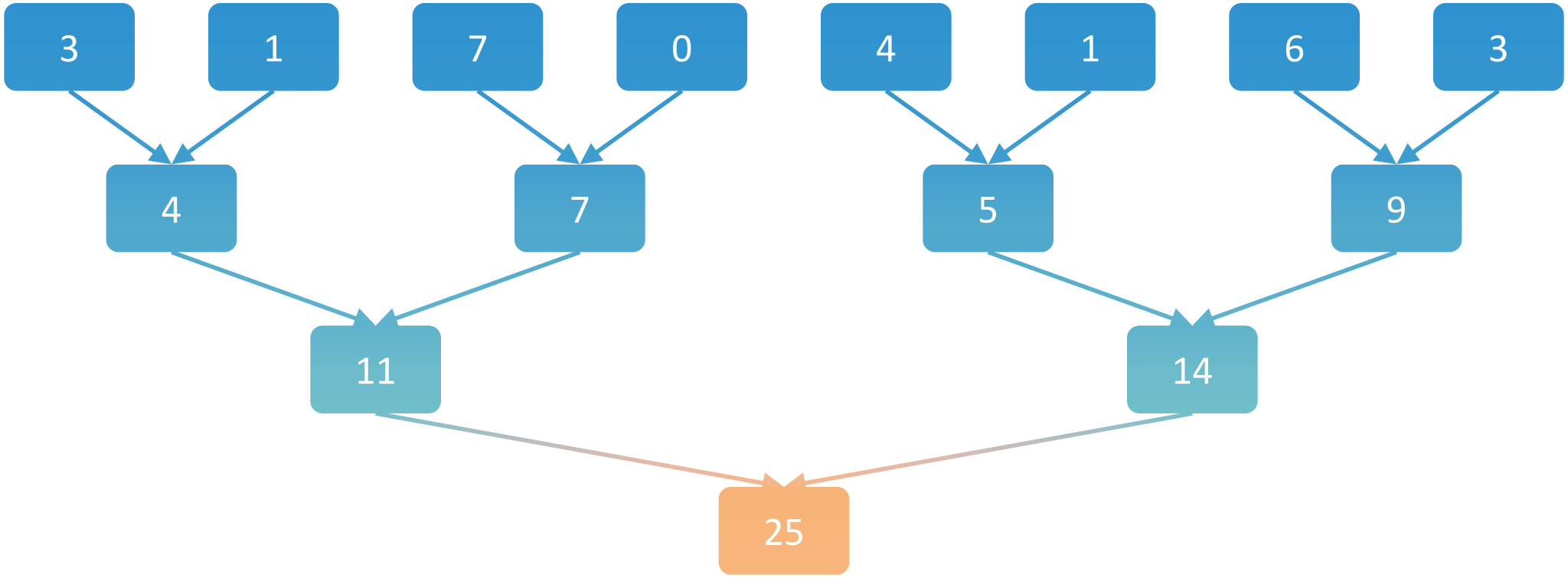
Evaluate each change against the same correct baseline.

06

Parallel Reduction

A Reduction Tree

上海创智学院
Shanghai Innovation Institute



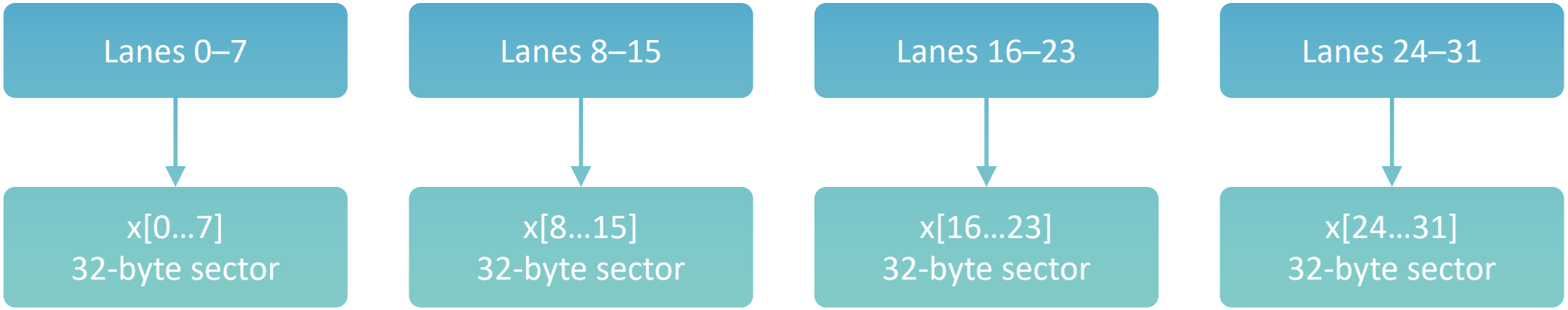
Independent additions within a level. Dependencies between levels.

Reduction: Coalesced Global Loads

上海创智学院
Shanghai Innovation Institute

```
int t = threadIdx.x, base = blockIdx.x * B;
s[t] = x[base + t];   // one load instruction across a warp
__syncthreads();
```

Warp 0: 32 lanes read 32 consecutive floats



4 sectors = 128 B per warp; full block B = 128 reads 512 B.

base = 0 shown; aligned FP32, CC ≥ 6.0 sector model. Both reduction loops can use this same global load.

Reduction: Interleaved Addressing

上海创智学院
Shanghai Innovation Institute

```
for (int stride = 1; stride < B;
     stride *= 2) {
  if (t % (2*stride) == 0)
    s[t] += s[t + stride];
  __syncthreads();
}
```

Active thread IDs are interleaved throughout the block.

All block threads reach each barrier.

B = 8; each row shows shared positions 0 … 7

Initial tile

3	1	7	0	4	1	6	3

stride = 1 · T0,2,4,6

4	–	7	–	5	–	9	–

stride = 2 · T0,4

11	–	–	–	14	–	–	–

stride = 4 · T0

25	–	–	–	–	–	–	–

Start each variant from the original tile. “–” marks unused old values; both variants produce 25.

Reduction: Divergent and Uniform Warps

上海创智学院
Shanghai Innovation Institute

```
// B = 128; compare the first stage
if (t % 2 == 0) s[t] += s[t+1];
// versus: if (t < 64) s[t] += s[t+64];
```

Each row is one full 32-thread warp; + participates, – skips the add

Interleaved W0



Interleaved W1



Prefix W0 / W1



Prefix W2 / W3



Interleaved: every warp mixes paths. Prefix: whole warps agree at this stage.

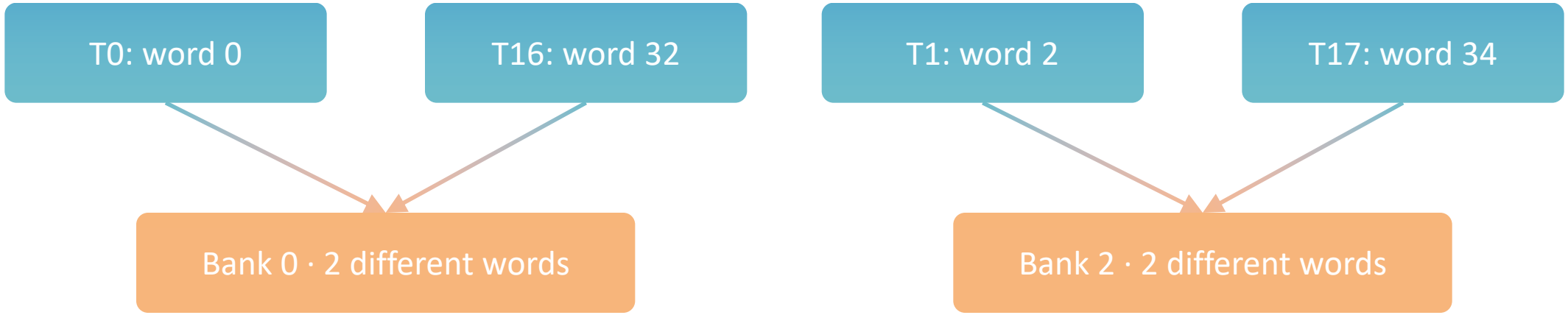
W2/W3 match the interleaved pattern too. Skipping an add does not skip the block barrier.

Reduction: Contiguous Threads, Strided Words

上海创智学院 Shanghai Innovation Institute

```
int index = 2 * stride * t;
if (index < B) s[index] += s[index + stride];
__syncthreads();
```

B = 128, stride = 1: contiguous threads, but words 0, 2, …, 62



One scalar FP32 shared load, 32 banks: word mod 32. A contiguous active prefix alone does not avoid conflicts.

Reduction: Sequential Addressing

```
for (int stride = B/2; stride > 0;
     stride /= 2) {
  if (t < stride)
    s[t] += s[t + stride];
  __syncthreads();
}
```

Active threads use consecutive shared addresses per load.

All block threads reach each barrier.

B = 8; each row shows shared positions 0 … 7

Initial tile

3	1	7	0	4	1	6	3

stride = 4 · T0–T3

7	2	13	3	–	–	–	–

stride = 2 · T0–T1

20	5	–	–	–	–	–	–

stride = 1 · T0

25	–	–	–	–	–	–	–

Start each variant from the original tile. “–” marks unused old values; both variants produce 25.

Reduction: Uniform Warps and the Tail

上海创智学院
Shanghai Innovation Institute

```
for (int stride = 64; stride > 0; stride /= 2) {
  if (t < stride) s[t] += s[t + stride];
  __syncthreads(); }
```

B = 128: number of lanes doing the add in each warp

stride	Warp 0	Warp 1	Warp 2	Warp 3
64	32	32	0	0
32	32	0	0	0
16	16	0	0	0

stride = 16
Warp 0



Whole warps agree in early stages; stride 16, 8, 4, 2, 1 leaves a partially active warp. Keep required barriers.

Block Reduction in Shared Memory

上海创智学院
Shanghai Innovation Institute

B = 4, shared s starts as [1,2,3,4] after the initialization barrier

```
for(int stride=B/2;stride>0;stride/=2) {
  if(t<stride) s[t]+=s[t+stride];
  __syncthreads();
}
if(t==0) out[blockIdx.x]=s[0];
```

Initial



stride = 2



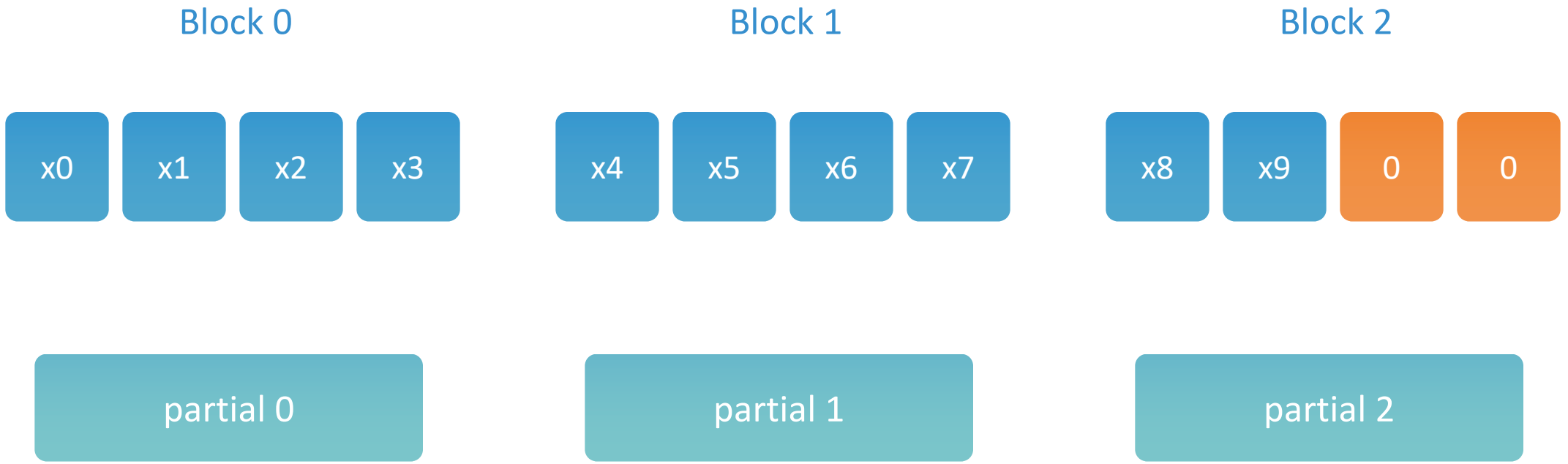
stride = 1



Only the active prefix continues.
Gray entries are no longer partial outputs.

Global: 4 input reads, 1 output write. Intermediate updates use shared memory and block barriers.

Tail Elements and Stage Sizes

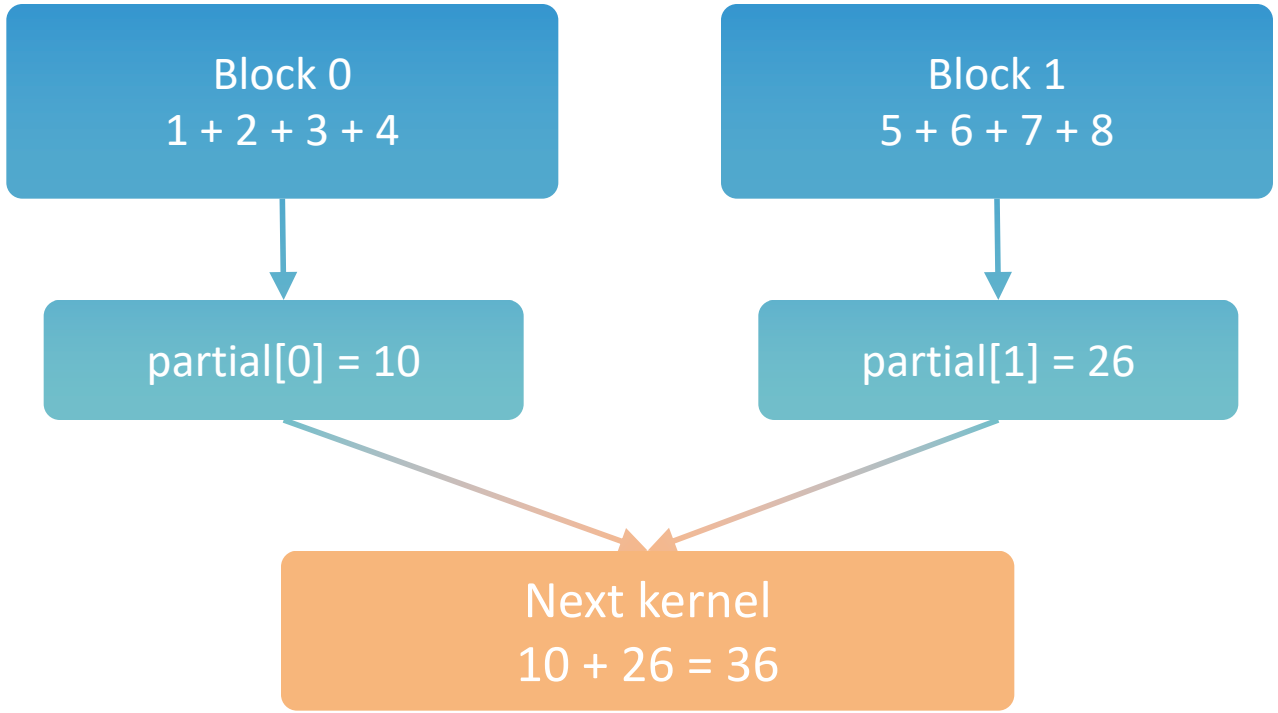


Next stage: [partial 0, partial 1, partial 2, 0]

n = 10, B = 4: 3 partial sums, then 1 result.

Partial Results Across Blocks

x = [1,2,3,4,5,6,7,8], B = 4



Kernel 1
8 global reads
2 global writes

Kernel 2
2 global reads
1 global write

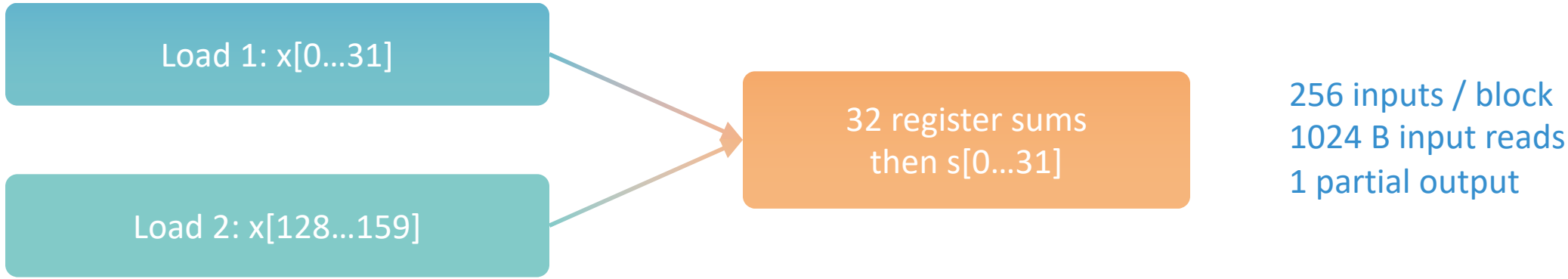
Same stream: reduce<<<2,4>>>(x,partial,8); then reduce<<<1,4>>>(partial,out,2).

Reduction: Two Inputs per Thread

上海创智学院
Shanghai Innovation Institute

```
int i = blockIdx.x * (2*B) + t;
float a = (i < n) ? x[i] : 0.0f;
float b = (i+B < n) ? x[i+B] : 0.0f;
s[t] = a + b;
__syncthreads();  // continue with the same shared tree
```

B = 128, block 0: each load is contiguous across warp 0



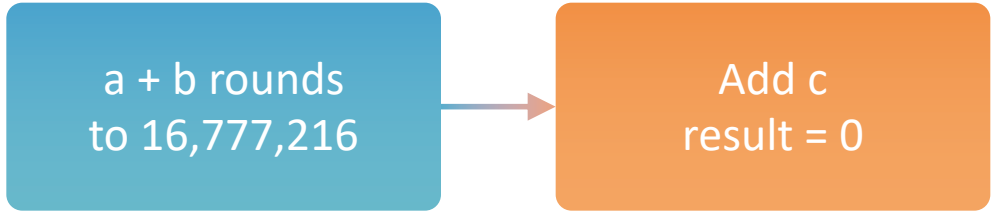
256 inputs / block
1024 B input reads
1 partial output

A full block, FP32. Still read every input; fewer blocks and partial sums do not guarantee a speedup.

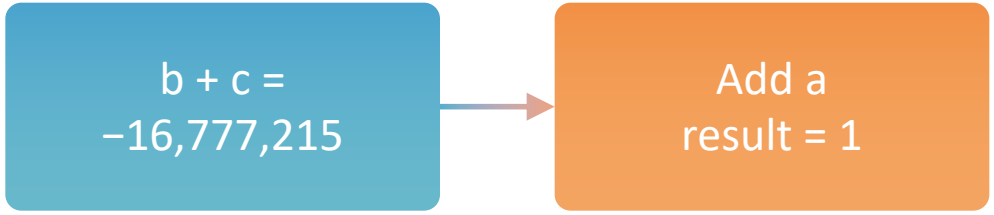
Floating-Point Addition Has an Order

FP32: a = 16,777,216; b = 1; c = −16,777,216

(a + b) + c



a + (b + c)



Exact arithmetic gives 1. Different groupings can round differently.

Round-to-nearest-even FP32. Validate with a suitable tolerance; still rule out races and bugs.

Architecture Recap

上海创智学院
Shanghai Innovation Institute

```
row = blockIdx.y*T + ty;
col = blockIdx.x*T + tx;
```

→ Output ownership

```
As[ty][tx] = A[row*K + k0 + tx];
```

→ Adjacent tx → adjacent loads

```
__syncthreads();
```

→ Block cooperation

```
sum += As[ty][kk] * Bs[kk][tx];
```

→ Private accumulator

T = 2 example: one block reduces global input requests from 128 B to 64 B.

Tiled GEMM excerpt. Match each code fragment to its architectural reason. Edge checks omitted.

Final Check: Explain the Optimization

上海创智学院
Shanghai Innovation Institute

A correct kernel becomes slower after adding a larger shared tile.

1. Which new costs or resource limits could explain this?
2. What would you measure before changing the code again?
3. Which architecture concepts support your explanation?

Use evidence to connect program behavior with architecture.

Acknowledgement

The development of this course, including its structure, content, and accompanying presentation slides, has been significantly influenced and inspired by the excellent work of instructors and institutions who have shared their materials openly. We wish to extend our sincere acknowledgement and gratitude to the following courses, which served as invaluable references and a source of pedagogical inspiration:

- Machine Learning Systems[15-442/15-642], by **Tianqi Chen** and **Zhihao Jia** at **CMU**.
- Advanced Topics in Machine Learning (Systems)[CS6216], by **Yao Lu** at **NUS**

While these materials provided a foundational blueprint and a wealth of insightful examples, all content herein has been adapted, modified, and curated to meet the specific learning objectives of our curriculum. Any errors, omissions, or shortcomings found in these course materials are entirely our own responsibility. We are profoundly grateful for the contributions of the educators listed above, whose dedication to teaching and knowledge-sharing has made the creation of this course possible.

System for Artificial Intelligence

Thanks

Siyuan Feng
Shanghai Innovation Institute